

ENTORNOS DE DESARROLLO

Sesión Experiencial

App en Python

con Arquitectura de Capas

APP de gestión de incidencias

Alumno: Francisco José García Franco

1º DAM - UAX

2025/2026

Sesión Experiencial – 25-04-2026**1. ENUNCIADO****2. ENFOQUE****3. SOLUCIÓN PROPUESTA POR EL PROFESOR****3.1 Arquitectura de capas****3.2 Esquema básico de la app / mapa de interconexión de archivos****3.3 Análisis detallado del código****3.3.1 models.py (Capa de Dominio)****3.3.2 db.py (Capa de Infraestructura)****3.3.3 repository.py (Capa de Acceso a Datos)****> Análisis de la clase TechnicianRepository**

- Método create
- Método find_all
- Método find_by_id
- Método map_row_to_technician

> Análisis de la clase TicketRepository

- Método create
- Método find_all
- Método find_by_id
- Método update_status
- Método assign_technician
- Método find_open_tickets
- Método map_row_to_ticket

3.3.4 service.py (Capa de Servicio / Capa de Lógica de Negocio)**> Métodos**

- De creación
 - Método register_technician
 - Método create_ticket
- De operación y validación de las reglas de negocio
 - Método assign_ticket
 - Método close_ticket
- De consulta
 - Método get_all_tickets
 - Método get_all_technician
 - Método get_open_tickets
 - Método get_ticket_by_id

3.3.5 app.py (Punto de Entrada)**> Análisis de funcionalidad****> Comprobaciones realizadas por el profesor (app.py)****> Propuesta de operaciones / comprobaciones (app2.py)****> Descripción de operaciones paso a paso**

- Inicialización de infraestructura
- Creación de entidades (Técnicos)
- Lectura de datos (todos los Técnicos)
- Creación de entidades (Tickets)
- Lectura filtrada (Tickets abiertos)
- Modificación/Regla de Negocio (Asignación de Tickets a Técnicos)
- Búsqueda por identificador (de Ticket)
- Modificación/Regla de Negocio (Cierre de Ticket)
- Lectura general (todos los Tickets)

4. COMPLETANDO EL CÓDIGO (Requerimientos del enunciado)**4.1 Implementando el código****4.2 Comprobaciones adicionales****> Notas****> raise ValueError**

1. ENUNCIADO

Contexto empresarial

La empresa TechSolutions S.L. necesita desarrollar un sistema interno para gestionar incidencias reportadas por sus empleados. Actualmente, el proceso se realiza por correo electrónico, lo que genera:

- falta de seguimiento
- duplicidad de incidencias
- mala asignación de técnicos
- ausencia de métricas

El equipo de desarrollo ha decidido construir una primera versión del backend en Python. Tu equipo será responsable de desarrollar este sistema.

Objetivo del proyecto

Construir una aplicación backend en Python que permita:

- registrar técnicos
- crear incidencias (tickets)
- asignar incidencias a técnicos
- gestionar el estado de las incidencias
- consultar información relevante

El sistema debe seguir buenas prácticas profesionales:

- tipado en Python
- acceso a base de datos
- separación por capas
- uso de control de versiones con Git y GitHub

Gestión de técnicos

El sistema debe permitir:

- Registrar un técnico con:
 - nombre
 - email
 - estado activo/inactivo
- Consultar todos los técnicos

Gestión de incidencias (tickets)

Cada ticket debe contener:

- título
- descripción
- email del creador
- estado:
 - OPEN
 - IN_PROGRESS
 - CLOSED
- técnico asignado (opcional)

Reglas de negocio (muy importante)

Debes implementar las siguientes validaciones:

- No se puede crear un ticket con título vacío o demasiado corto
- El email del creador debe ser válido (contener @)
- No se puede asignar un ticket a un técnico inexistente
- No se puede asignar un ticket a un técnico inactivo
- No se puede cerrar un ticket ya cerrado
- (Nivel medio) No se puede cerrar un ticket sin técnico asignado

2. ENFOQUE

Se pretende implantar una app de gestión de incidencias mediante la conexión a una BBDD embebida (SQLite). Se trata de un proyecto de desarrollo de un sistema **backend** clásico, bien acotado y estructurado, ideal para asentar conceptos de:

- Bases de datos
- Arquitectura de software
- Lógica de negocio.

Análisis desglosado de los requisitos y características del proyecto:

1. Contexto y Objetivo del Proyecto

- **El Problema:** la empresa TechSolutions S.L. gestiona sus incidencias por correo electrónico, lo que provoca falta de seguimiento, duplicidad, mala asignación y ausencia de métricas.
- **La Solución:** construir la primera versión de una aplicación backend en Python para gestionar de forma centralizada estos reportes o *tickets*.

2. Requisitos Técnicos y Arquitectura

El sistema exige implementar buenas prácticas de la industria para garantizar un código mantenible:

- **Lenguaje y Tipado:** desarrollo en Python utilizando **Type Hinting** (tipado estático opcional) para mayor claridad.
- **Persistencia de Datos:** conexión a una base de datos (se ha optado por SQLite embebida).
- **Diseño:** uso de una arquitectura basada en la **separación por capas**; típicamente:
 - Rutas/controladores.
 - Servicios/lógica de negocio.
 - Repositorios/datos.
- **Control de Versiones:** uso de Git y GitHub para la gestión del código.

3. Modelo de Datos (Entidades Principales)

El sistema gira en torno a dos entidades clave:

- **Técnico:** requiere almacenar su nombre, email y un estado que indique si está activo o inactivo.
- **Ticket (Incidencia):** debe almacenar un título, una descripción, el email del empleado que crea la incidencia, su estado (OPEN, IN_PROGRESS, CLOSED) y, de forma opcional, el técnico que se le asigne.

4. Operaciones del Sistema (Casos de Uso)

El backend debe exponer funcionalidades específicas para ambas entidades:

- **Para Técnicos:** capacidad para registrar nuevos técnicos y consultar el listado completo de los mismos.
- **Para Tickets:** capacidad para crearlos, asignarlos a un técnico, modificar su estado, cerrarlos y realizar consultas (ver todos los tickets o filtrar únicamente los que están abiertos).

5. Reglas de Negocio (Validaciones Críticas)

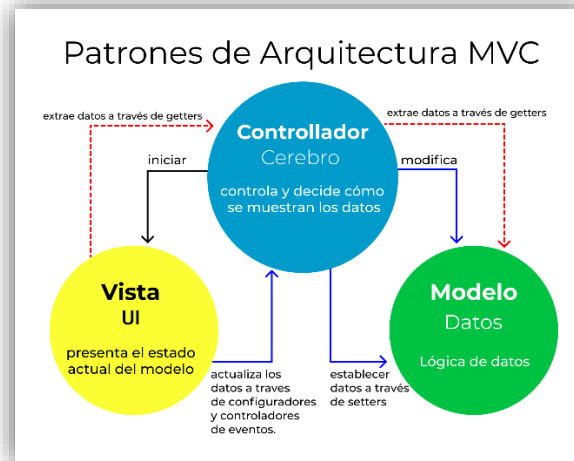
Esta es la parte central de la lógica del sistema. Se deben bloquear ciertas acciones para mantener la integridad de los datos:

- **Validación de entrada:** el título del ticket no puede estar vacío ni ser demasiado corto, y el email del creador debe contener una "@".
- **Validación de asignaciones:** no se puede asignar un ticket a un técnico que no existe en la base de datos o que tiene un estado inactivo.
- **Validación de estado:** un ticket que ya está en estado CLOSED no puede volver a cerrarse. Además, se prohíbe cerrar un ticket si previamente no se le ha asignado un técnico.

3. SOLUCIÓN PROPUESTA POR EL PROFESOR

3.1 Arquitectura de capas

> EL profesor inicialmente menciona una "lógica **MVC**" (**Modelo-Vista-Controlador**):



- > **Modelo** → los datos: se define el tipo de datos dentro de la app.
- > **Controlador** → la lógica de la app.
- > **Vista** → lo que ve el usuario.

Este enfoque, en el que el código se divide en capas que se conectan entre si mediante POO, permite:

- Mucha mayor fluidez dentro de la app:
 - En caso de Big Data la carga será mucho más rápida.
 - Si se muestran muchas imágenes se evita que se traben.
- Sea más fácil buscar errores.

> La realidad es que la solución propuesta se alinea más concretamente con una **Arquitectura en Capas (Layered Architecture)** o el patrón **Servicio-Repositorio**, que es el estándar ideal para backends modernos.

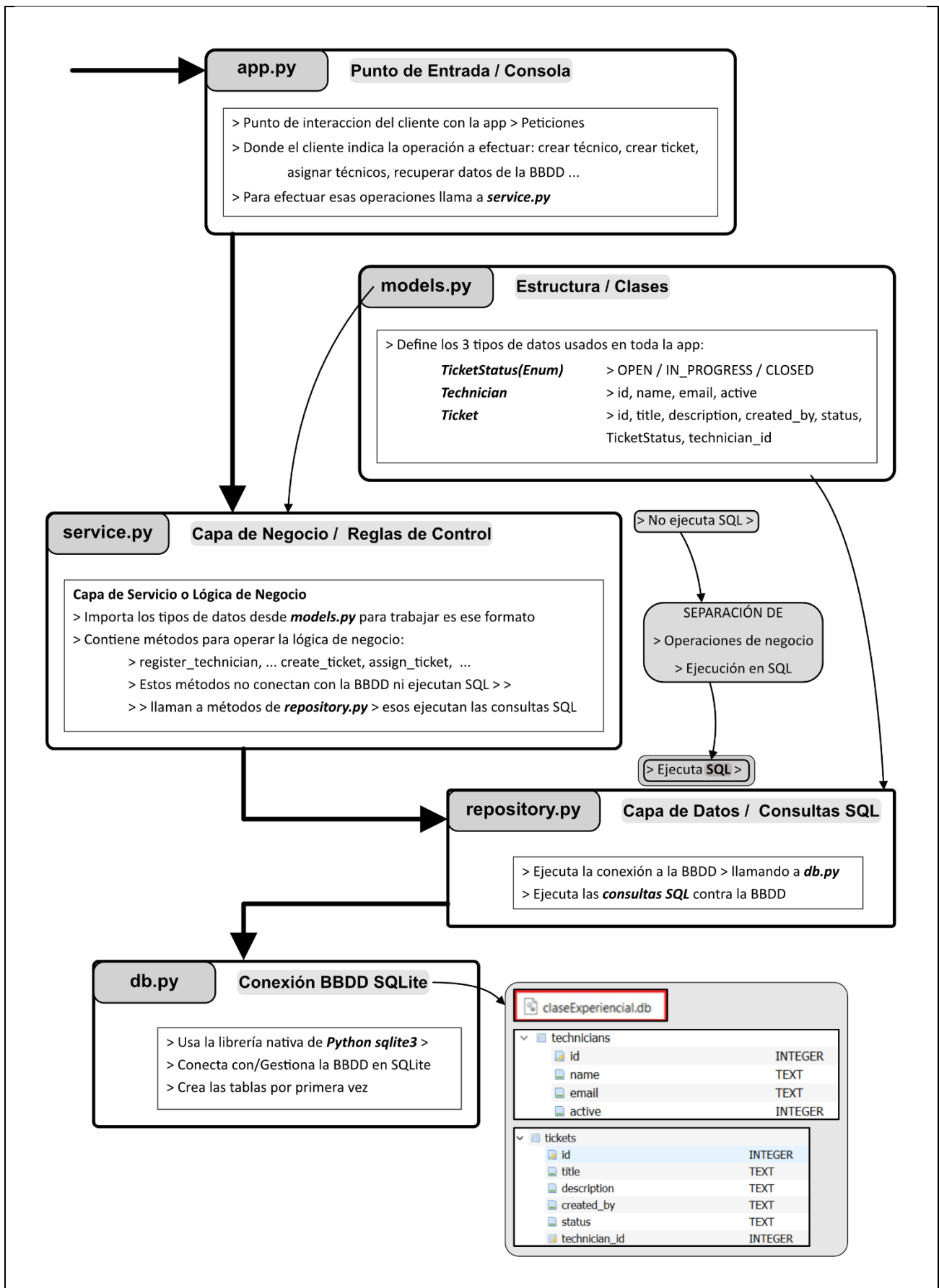
Análisis de estructura del código > Estructura y Separación de Responsabilidades

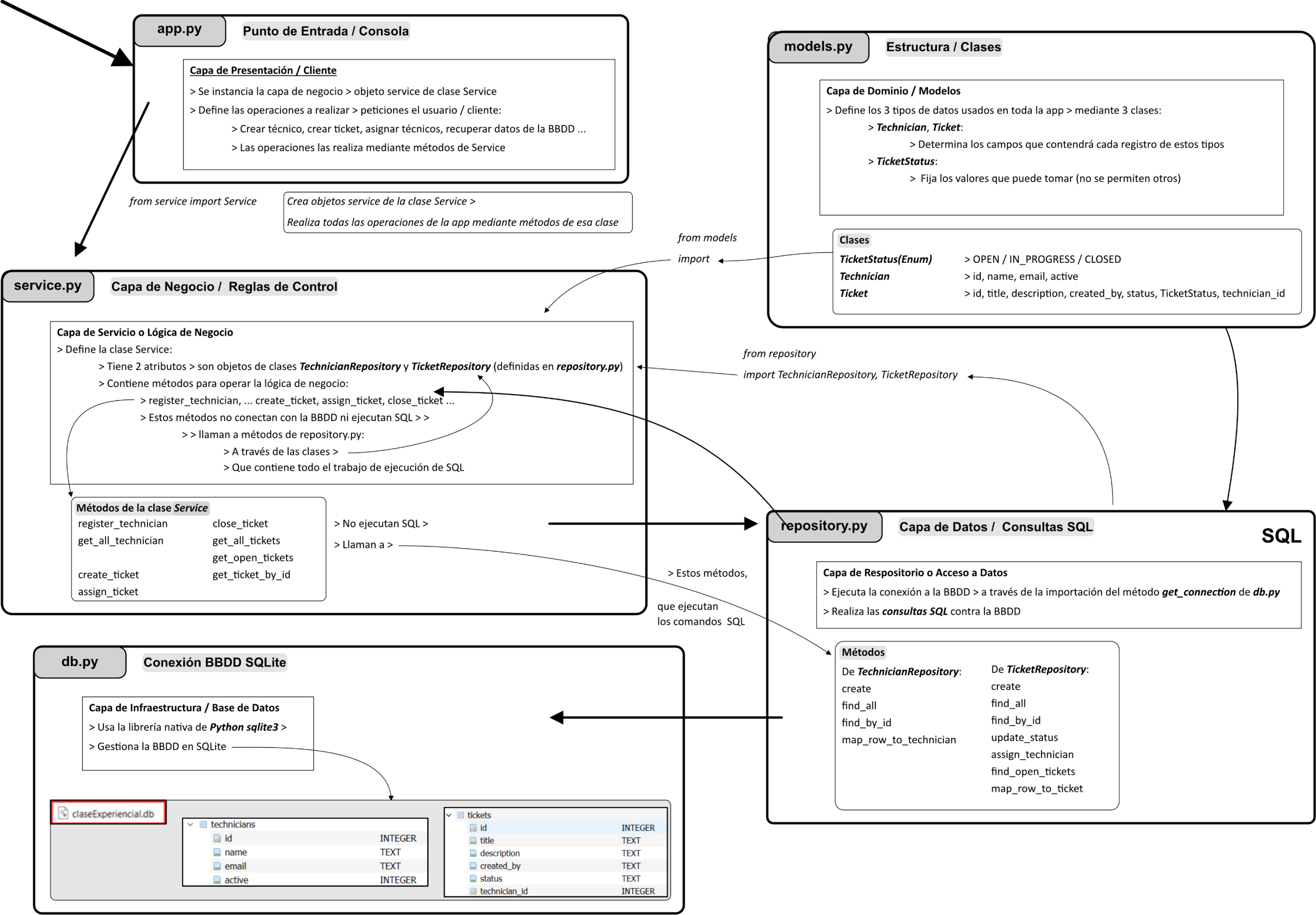
Cada archivo tiene una responsabilidad única:

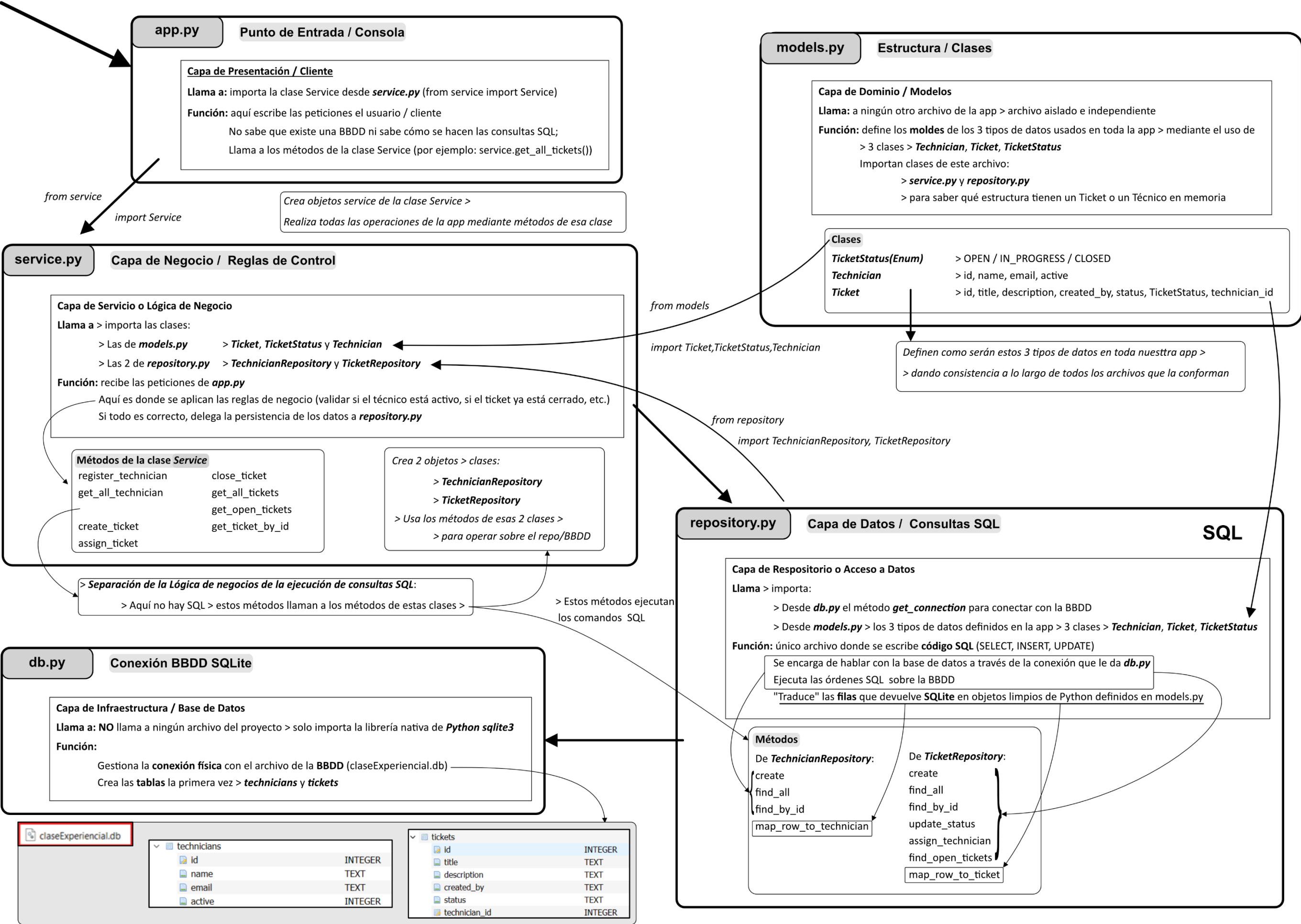
- **models.py (Dominio):** define la forma de los datos usando **dataclasses** (muy buena práctica en Python moderno) y **Enum** para los estados (TicketStatus). Representa la capa más pura, sin dependencias externas.
- **db.py (Infraestructura):** centraliza la conexión a SQLite y la creación de las tablas (**init_db()**). Cumple perfectamente con el esquema relacional requerido (clave foránea de technician_id en tickets).
- **repository.py (Capa de Acceso a Datos):** aísla por completo las consultas SQL (**CRUD**). Si mañana la empresa decide cambiar de SQLite a PostgreSQL, *solo* habría que tocar este archivo. Hace un buen trabajo mapeando las filas (Row) a los objetos del modelo.
- **service.py (Capa de Lógica de Negocio):** es el "cerebro" de la aplicación; gestiona las llamadas a los repositorios y (teóricamente) aplica las reglas de negocio.
- **app.py (Presentación / Entrypoint):** actúa como cliente o "Vista/Controlador" provisional para probar que la lógica funciona.

3.2 Esquema básico de la app / mapa de interconexión de archivos

Para entender cómo interactúan los archivos entre sí y cómo fluye la información en esta arquitectura de capas (Estructura Servicio-Repositorio), se puede representar como una cadena de dependencias que va desde la parte más externa (la interacción con el usuario) hasta la más interna (la base de datos).







Explicación del flujo:**1. app.py (Capa de Presentación / Cliente):**

- **A quién llama:** únicamente importa y llama a **service.py** (from service import Service).
- **Función:** permite la interacción del cliente; aquí puede escribir las peticiones a la app. Este archivo no sabe que existe una base de datos ni sabe cómo se hacen las consultas SQL; solo hace peticiones al "Servicio" (por ejemplo: service.get_all_tickets()).

2. service.py (Capa de Servicio o Lógica de Negocio):

- **A quién llama:** importa clases desde los archivos **repository.py** y a **models.py**.
- **Función:** recibe las peticiones de app.py. Aquí es donde se aplican las reglas de negocio (validar si el técnico está activo, si el ticket ya está cerrado, etc.). Si todo es correcto, delega la persistencia de los datos al "Repositorio".

3. repository.py (Capa de Repositorio o Acceso a Datos):

- **A quién llama:** importa métodos desde **db.py** (from db import **get_connection**) y clases desde **models.py**.
- **Función:** es el único archivo donde se escribe **código SQL** (**SELECT, INSERT, UPDATE**). Se encarga de hablar con la base de datos a través de la conexión que le da **db.py** y de "traducir" las filas que devuelve SQLite en objetos limpios de Python definidos en **models.py**.

4. db.py (Capa de Infraestructura / Base de Datos):

- **A quién llama:** no importa nada desde ningún archivo del proyecto. Solo usa la librería nativa de Python **sqlite3**.
- **Función:** su único trabajo es gestionar la conexión física con el archivo de la base de datos (**claseExperiencial.db**) y crear las tablas la primera vez.

5. models.py (Capa de Dominio / Modelos):

- **A quién llama:** a ningún archivo; es completamente aislado e independiente.
- **Función:** Define los moldes de los datos (**Technician, Ticket, TicketStatus**) como clases. Esas clases las importan tanto el **Servicio** (service.py) como el **Repositorio** (repository.py) para saber qué estructura tienen un Ticket o un Técnico en memoria.

Esta estructura evita el "código espagueti". Si el día de mañana se decide cambiar la base de datos de SQLite a MySQL, "solo" habría que modificar **db.py** y **repository.py**. El resto de la aplicación (app.py y service.py) seguiría funcionando exactamente igual sin enterarse del cambio.

3.3 Análisis detallado del código

A continuación, se realiza un estudio de **dentro hacia fuera** (**enfoque** conocido como **Bottom-Up**).

Se comienza por el análisis de los archivos más puros (los que no dependen de ningún otro) y se termina por los más complejos (los que coordinan a los demás).

Este es el **orden de estudio** de los archivos y la **razón**:

1. **models.py (El corazón de los datos)**: aquí se define qué es un Técnico y qué es un Ticket dentro de la app. No se importa ningún otro archivo; es 100% independiente. Se da forma a los tipos de datos del problema a resolver.
2. **db.py (La infraestructura física)**: seguidamente se estudia cómo se ha configurado la creación y conexión de la base de datos SQLite y cómo se relacionan las tablas por debajo.
3. **repository.py (El traductor)**: aquí se unen los dos pasos anteriores. Este archivo coge las filas de la base de datos (**db.py**) y las convierte en los objetos de Python que vimos al principio (**models.py**).
4. **service.py (El cerebro de la operación)**: es la capa crítica. Aquí se aplican las reglas de negocio y se coordina todo.
5. **app.py (El punto de entrada)**: aquí se arranca la aplicación y un usuario (o un sistema externo) puede hacer peticiones a **service.py**.

3.3.1 *models.py*

Este archivo pertenece a la **Capa de Dominio**.

Su único objetivo es definir las estructuras de datos (los "moldes") con los que operará la aplicación en memoria. Lo interesante es que no tiene código que guarde datos en archivos ni código que muestre pantallas; es puramente representativo.

```
from enum import Enum
from dataclasses import dataclass
from typing import Optional

class TicketStatus(Enum):
    OPEN = "OPEN"
    IN_PROGRESS = "IN_PROGRESS"
    CLOSED = "CLOSED"

@dataclass
class Technician:
    id: Optional[int]
    name: str
    email: str
    active: bool = True

@dataclass
class Ticket:
    id: Optional[int]
    title: str
    description: str
    created_by: str
    status: TicketStatus
    technician_id: Optional[int] = None
```

> Importaciones

```
from enum import Enum
from dataclasses import dataclass
from typing import Optional
```

- **Enum**: sirve para crear un conjunto de constantes con nombre.
- **dataclass**: decorador de Python (introducido en Python 3.7) que **genera automáticamente métodos estándar para las clases**, como el **constructor** (`__init__`) y el **formateo** para imprimir en **pantalla** (`__repr__`), ahorrando código repetitivo.
- **Optional**: Se usa en el tipado para indicar que una variable puede tener un tipo concreto (como **int**) o bien valer **None**.

> Enumerado: TicketStatus

```
class TicketStatus(Enum):
    OPEN = "OPEN"
    IN_PROGRESS = "IN_PROGRESS"
    CLOSED = "CLOSED"
```

En lugar de usar textos sueltos (*strings*) a lo largo del código (como "abierto", "en_proceso", "cerrado"), lo cual es peligroso porque un error de dedo como "open" rompería la aplicación, se usa un **Enum**; se llama a los nombres cerrados como **métodos** de la clase, pudiendo elegir solo los de la lista. Así se hace referencia a los estados de forma segura con **TicketStatus.OPEN** o **TicketStatus.CLOSED** ... sin que pueda haber posibles errores.

> Modelo de dato > **Technician** (Técnico)

```
@dataclass
class Technician:
    id: Optional[int]
    name: str
    email: str
    active: bool = True
```

Gracias a **@dataclass**, no hace falta escribir **def __init__(self, id, name...)**; Python lo crea por detrás.

- **id: Optional[int]: Esto es clave.** ¿Por qué es opcional? Al registrar un técnico por primera vez en el programa, *aún no tiene ID* (vale None). El ID numérico se lo asignará automáticamente la base de datos (SQLite) una vez que lo insertemos.
- **name y email:** son cadenas de texto obligatorias.
- **active: bool = True:** por defecto, cualquier técnico que se cree en el sistema se registrará como "activo" (True), a menos que se indique lo contrario.

> Modelo de dato > **Ticket** (Incidencia)

```
@dataclass
class Ticket:
    id: Optional[int]
    title: str
    description: str
    created_by: str
    status: TicketStatus
    technician_id: Optional[int] = None
```

Representa una incidencia en el sistema y se conecta directamente con el Técnico:

- **id:** al igual que en el caso del técnico, es opcional porque SQLite lo genera al guardar el ticket.
- **status: TicketStatus:** se fuerza a que el estado del ticket pertenezca obligatoriamente al enumerado creado al principio (OPEN, IN_PROGRESS o CLOSED) > clase **TicketStatus**
- **technician_id: Optional[int] = None:** campo que guarda el ID del técnico asignado a la incidencia. Es opcional y su valor por defecto es None porque, tal y como dice el enunciado, un ticket cuando nace se crea sin ningún técnico asignado (estado OPEN).

> > **Resumen de models.py**

Este archivo define el **lenguaje común** de la aplicación. Cuando la base de datos devuelva datos, el repositorio los transformará en estas clases. Cuando el archivo **app.py** pida información, recibirá estas **clases**.

3.3.2 *db.py*

Este archivo representa la **Capa de Infraestructura**.

Su única misión es comunicarse con el sistema de archivos del ordenador para gestionar la base de datos física (SQLite) y crear la estructura de tablas necesaria para el proyecto.

```
import sqlite3
from sqlite3 import Connection

DB_NAME = "claseExperiencial.db"

def get_connection() -> Connection:
    conn = sqlite3.connect(DB_NAME)
    return conn

def init_db() -> None:
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute("""
    CREATE TABLE technicians (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        email TEXT NOT NULL UNIQUE,
        active INTEGER NOT NULL CHECK (active IN (0,1))
    )
    """)

    cursor.execute("""
    CREATE TABLE tickets (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        description TEXT NOT NULL,
        created_by TEXT NOT NULL,
        status TEXT NOT NULL,
        technician_id INTEGER,
        FOREIGN KEY (technician_id) REFERENCES technicians(id)
    )
    """)

    conn.commit()
    conn.close()
```

> Configuración y Conexión

```
import sqlite3
from sqlite3 import Connection

DB_NAME = "claseExperiencial.db"

def get_connection() -> Connection:
    conn = sqlite3.connect(DB_NAME)
    return conn
```

- **DB_NAME = "claseExperiencial.db"**: define el nombre del archivo donde se guardarán los datos. Si este archivo no existe en la carpeta del proyecto, SQLite lo creará automáticamente la primera vez que se ejecute el código.
- **get_connection()**: esta función abre una conexión con el archivo de la base de datos y la devuelve (la salida es un objeto de tipo conexión a BBDD). Es fundamental porque **será llamada continuamente por los repositorios** cada vez que necesiten leer o escribir datos. Al centralizarla aquí, cualquier cambio futuro en el nombre de la base de datos solo implicará la modificación de esta línea.

> def get_connection() -> Connection:

Esta línea es la **cabecera** o la declaración de la función.

- **def:** palabra clave (*keyword*) para *definir una nueva función*.
- **get_connection:** nombre elegido para la función; nombre muy descriptivo que significa "obtener conexión".
- **():** Los paréntesis vacíos significan que esta función **no necesita parámetros ni argumentos** de entrada para trabajar.
- **-> Connection:** Esto es un **indicador de tipo (Type Hint)**.
No cambia cómo se ejecuta el programa, pero sirve para documentar el código. Indica que cuando esta función termine devolverá un objeto de tipo Connection (el cual se importó de la librería sqlite3).

> conn = sqlite3.connect(DB_NAME)

Esta línea abre la comunicación.

- **sqlite3:** referencia al módulo nativo de Python importado más arriba en el archivo y que contiene todas las herramientas para manejar bases de datos SQLite.
- **.connect(...):** método del módulo sqlite3.
Se dirige al disco duro del ordenador, busca un archivo de base de datos e inicia un canal de comunicación seguro y directo con él.
- **(DB_NAME):** argumento que se pasa a la función **.connect()**.
DB_NAME es una constante que vale el nombre que le damos a nuestra BBDD: **"claseExperiencial.db"**.
Se le está pidiendo a SQLite que se conecte a este archivo.
Si el archivo "claseExperiencial.db" no existe en la carpeta en ese momento, la función .connect() no da error; lo crea vacío automáticamente.
- **conn =:** variable local a la que se le da el nombre de conn (abreviatura de *connection*); almacenará el canal de comunicación que acaba de abrir .connect(). A partir de este momento, el objeto conn representa la BBDD activa en la memoria de Python.

> return conn

Esta línea es la **salida** de la función.

- **return:** palabra clave que finaliza la ejecución de la función y ordena devolver el objeto a quien lo haya llamado.
- **conn:** objeto de conexión creado en la línea anterior y que se devuelve.

¿Por qué es necesario el return? Porque cuando un repositorio (**repository.py**) necesite hacer un INSERT o un SELECT, escribirá algo como `conexion_activa = get_connection()`. Gracias al `return conn`, esa variable del repositorio recibirá el canal de comunicación listo para enviar comandos SQL.

> Inicialización de las Tablas: `init_db()`

Esta función se encarga de estructurar el modelo relacional ejecutando **código SQL a través de un cursor** (el objeto que envía las órdenes a la base de datos). Crea dos tablas que encajan con los modelos que vimos en el archivo anterior:

A) La Tabla de Técnicos (`technicians`)

```
CREATE TABLE technicians (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    name TEXT NOT NULL,  
    email TEXT NOT NULL UNIQUE,  
    active INTEGER NOT NULL CHECK (active IN (0,1))  
)
```

- **id INTEGER PRIMARY KEY AUTOINCREMENT:** define la clave primaria. Cada vez que se inserte un técnico, SQLite le asignará un número único (1, 2, 3...) de forma automática. Aquí cobra sentido que en `models.py` el ID fuera **opcional** (Optional[int]).
- **email TEXT NOT NULL UNIQUE:** obliga a que el email no esté vacío y añade la restricción UNIQUE. El sistema lanzará un error si se intenta registrar a dos técnicos con el mismo correo.
- **active INTEGER NOT NULL CHECK...:** **Ojo con este detalle.** SQLite no tiene un tipo de datos "Booleano" nativo (True/False). Para solucionar esto se guarda un entero (INTEGER) y se añade una restricción (CHECK) para asegurar que solo se puedan introducir los valores 0 (Falso/Inactivo) o 1 (Verdadero/Activo).

B) La Tabla de Tickets o Incidencias (`tickets`)

```
CREATE TABLE tickets (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    title TEXT NOT NULL,  
    description TEXT NOT NULL,  
    created_by TEXT NOT NULL,  
    status TEXT NOT NULL,  
    technician_id INTEGER,  
    FOREIGN KEY (technician_id) REFERENCES technicians(id)  
)
```

- **Campos básicos:** almacena el título, descripción, el creador y el estado como texto obligado (NOT NULL).
- **technician_id INTEGER:** este campo guarda el ID del técnico asignado a la incidencia. Es opcional (no lleva NOT NULL), por lo que permite valores vacíos (NULL) cuando un ticket acaba de ser creado y nadie lo está atendiendo aún.
- **FOREIGN KEY (technician_id) REFERENCES technicians(id):** es la **Clave Foránea**. Es la línea que conecta ambas tablas de forma segura. Prohíbe a la base de datos asignar un ticket a un `technician_id` que no exista previamente en la tabla de técnicos

> **Análisis de la función `init_db()`:****Definición de la cabecera:**

- **`def init_db()` -> **None**:** declaración de la función orientada a la inicialización de la estructura de la base de datos.
El indicador de tipo determina que la función ejecuta procedimientos internos sin retornar ningún valor u objeto al finalizar.

Flujo secuencial de ejecución:

- **`conn = get_connection()`:** invocación de la función de conexión.
Asigna el objeto de conexión devuelto a la variable encargada de canalizar las operaciones sobre el archivo de la base de datos.
- **`cursor = conn.cursor()`:** creación de un objeto de tipo ***Cursor*** a partir de la conexión activa.
El cursor es la interfaz de control necesaria para ***enviar sentencias SQL individuales*** y procesar las modificaciones.
Sin este objeto, el sistema no puede interactuar con el motor de la base de datos.
- **`cursor.execute()`:** método principal del objeto cursor encargado de la transmisión y ejecución de sentencias SQL.
 - > Recibe como parámetro una cadena de texto que contiene el código SQL.
 - El motor de SQLite compila, valida la sintaxis y ejecuta dicha instrucción inmediatamente.
 - En este bloque se emplea para procesar de forma secuencial las instrucciones de creación de las tablas de técnicos y tickets.
- **`conn.commit()`:** operación de ***persistencia definitiva***.
En los sistemas relacionales, las modificaciones se procesan inicialmente dentro de una transacción temporal en memoria.
Este método confirma y consolida todos los cambios realizados a través del cursor, grabándolos permanentemente en el almacenamiento físico.
- **`conn.close()`:** procedimiento de cierre de recursos.
Corta la comunicación con el archivo de la base de datos y libera la memoria asignada a la conexión.
Es un paso necesario para evitar el bloqueo del archivo en el sistema operativo y prevenir errores de concurrencia

> **Cierre de la Operación**

```
conn.commit()
conn.close()
```

- **`conn.commit()`:** en las bases de datos, los cambios (como crear tablas o insertar filas) se quedan en una memoria temporal hasta que se da la orden de confirmación.
El método `commit()` graba de forma definitiva los cambios en el archivo del disco duro.
- **`conn.close()`:** cierra la conexión física para liberar los recursos del sistema

> > **Resumen de `db.py`**

Este archivo deja el terreno completamente preparado (***crea el archivo .db y sus tablas***) para que la aplicación empiece a funcionar.

> Con los moldes preparados en memoria (`models.py`) y las tablas en el disco (`db.py`), el siguiente paso es ver el archivo que sirve de "puente" o traductor entre ambos: **`repository.py`**

3.3.3 repository.py

Este archivo conforma la **Capa de Acceso a Datos (Data Access Layer)**.

Su función estructural es **aislar las operaciones de lectura y escritura**, cumpliendo con el requisito de acceso a base de datos y manteniendo una estricta separación por capas.

Transforma las filas devueltas por SQLite en objetos de Python (las clases definidas en *models.py*).

```
from models import Technician, Ticket, TicketStatus
from db import get_connection
from typing import List, Optional
from sqlite3 import Row

#INSERTAR
#LISTAR
#TOMAR UNA FILA Y PASARLO A UN TICKET

class TechnicianRepository:
    def create(self, technician: Technician):
        conn = get_connection()
        cursor = conn.cursor()

        cursor.execute(
            "INSERT INTO technicians (name, email, active) VALUES (?, ?, ?)",
            (technician.name, technician.email, 1 if technician.active else 0)
        )

        conn.commit()

        technician_id = cursor.lastrowid

        conn.close()

        return Technician(
            id= technician_id,
            name=technician.name,
            email=technician.email,
            active=technician.active
        )

    def find_all(self) -> List[Technician]:
        conn = get_connection()
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM technicians")
        rows = cursor.fetchall()
        conn.close()
        return [self.map_row_to_technician(row) for row in rows]

    def find_by_id(self, technician_id:int) -> Optional[Technician]:
        conn = get_connection()
        cursor = conn.cursor()
        cursor.execute(
            "SELECT * FROM technicians WHERE id = ?",
            (technician_id,)
        )
        row = cursor.fetchone()
        if row is None:
            return None
        conn.close()
        return self.map_row_to_technician(row)

    def map_row_to_technician(self, row: Row) -> Technician:
        return Technician(
            id = row[0],
            name=row[1],
            email=row[2],
            active=bool(row[3])
        )

class TicketRepository:
    def create(self, ticket: Ticket):
        conn = get_connection()
        cursor = conn.cursor()
```

```
cursor.execute(
    "INSERT INTO tickets (title, description, created_by,status,technician_id) VALUES (?, ?, ?, ?, ?)",
    (ticket.title, ticket.description, ticket.created_by,ticket.status.value, ticket.technician_id)
)

conn.commit()

ticket_id = cursor.lastrowid

conn.close()

return Ticket(
    id = ticket_id,
    title=ticket.title,
    description=ticket.description,
    created_by=ticket.created_by,
    status=ticket.status,
    technician_id=ticket.technician_id
)

def find_all(self) -> List[Ticket]:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM tickets")
    rows = cursor.fetchall()
    conn.close()
    return [self.map_row_to_ticket(row) for row in rows]

def find_by_id(self, ticket_id:int) -> Optional[Ticket]:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT * FROM tickets WHERE id = ?",
        (ticket_id,)
    )
    row = cursor.fetchone()
    if row is None:
        return None
    conn.close()
    return self.map_row_to_ticket(row)

def update_status(self, ticket_id:int, status: TicketStatus) -> bool:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        "UPDATE tickets SET status = ? WHERE id = ?",
        (status.value, ticket_id)
    )
    conn.commit()
    update = cursor.rowcount > 0
    conn.close()
    return update

def assign_technician(self, ticket_id: int, technician_id:int) -> bool:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        "UPDATE tickets SET technician_id = ?, status = ? WHERE id = ?",
        (technician_id, TicketStatus.IN_PROGRESS.value, ticket_id)
    )
    conn.commit()
    update = cursor.rowcount > 0
    conn.close()
    return update
```

```
def find_open_tickets(self) -> List[Ticket]:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("""
        SELECT *
        FROM tickets
        WHERE status = ?
    """, (TicketStatus.OPEN.value, ))
    rows = cursor.fetchall()
    conn.close()

    return [self.map_row_to_ticket(row) for row in rows]
def map_row_to_ticket(self, row: Row) -> Ticket:
    print(row[0])
    return Ticket(
        id = row[0],
        title=row[1],
        description=row[2],
        created_by=row[3],
        status=TicketStatus(row[4]),
        technician_id=row[5]
    )
```

> Importaciones específicas

- **`from typing import List, Optional`**: importación de módulos de tipado estático. **`List`** se emplea para indicar que una función retornará una colección de objetos. **`Optional`** señala que el retorno puede ser un objeto específico o **`None`** (empleado cuando una búsqueda en la base de datos no produce resultados).
- **`from sqlite3 import Row`**: importación del tipo de dato nativo que representa una fila de resultados obtenida tras ejecutar una consulta **`SELECT`**.
- **`from models import Technician, Ticket, TicketStatus`**
`from db import get_connection`

La importación de estos elementos en `repository.py` responde a la **necesidad de conectar la infraestructura de la base de datos con las estructuras de datos en memoria**, permitiendo al repositorio cumplir su función de capa de acceso a datos.

> Importación desde `models.py` (**`Technician, Ticket, TicketStatus`**)

El repositorio tiene como responsabilidad transformar los registros de la base de datos (filas y columnas) en objetos de Python. Para lograrlo, requiere conocer la estructura de dichos objetos:

- **`Technician` y `Ticket`**: son las clases necesarias para instanciar los objetos del dominio. Cuando el repositorio ejecuta una consulta de lectura (**`SELECT`**), extrae los datos crudos de SQLite y los introduce en los constructores de estas clases para devolver objetos estructurados. En operaciones de escritura (**`INSERT`** o **`UPDATE`**), el repositorio lee las propiedades de estos objetos para persistir los valores correctos en las tablas.
- **`TicketStatus`**: es el enumerado que gestiona los estados válidos de las incidencias. Se importa porque el repositorio debe traducir de forma segura las cadenas de texto almacenadas en la base de datos (como **`"OPEN"`**, **`"IN_PROGRESS"`** o **`"CLOSED"`**) a tipos enumerados nativos de Python, evitando errores de tipografía y garantizando la integridad de los datos durante las consultas y actualizaciones de estado.

> Importación desde `db.py` (**`get_connection`**)

El repositorio contiene la lógica de las sentencias SQL, pero no la configuración física de acceso al archivo de la base de datos.

- **`get_connection`**: se importa para que los métodos del repositorio puedan solicitar un canal de comunicación activo con SQLite en el momento exacto en que necesitan operar. Esto evita la duplicación de código relacionada con la apertura de conexiones en cada función y mantiene la **independencia de la capa de infraestructura** centralizada en **`db.py`**.

En conclusión, estas importaciones proveen al repositorio de los dos componentes esenciales para su funcionamiento:

- > El **mecanismo** para interactuar con el almacenamiento físico (**`get_connection`**).
- > Los **moldes lógicos** para estructurar la información procesada (**`Technician, Ticket, TicketStatus`**).

> Estructura de Clases

El archivo se divide en dos clases principales para mantener la cohesión lógica:

- **TechnicianRepository**: encapsula todas las **transacciones SQL** asociadas a la tabla **technicians**.
- **TicketRepository**: encapsula todas las **transacciones SQL** asociadas a la tabla **tickets**.

Ambas clases siguen un patrón de ejecución recurrente en sus métodos (Abrir conexión → Ejecutar consulta → Confirmar/Recuperar datos → Cerrar conexión → Retornar objeto).

> Análisis de la clase **TechnicianRepository**

Método **create**:

```
def create(self, technician: Technician):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute(
        "INSERT INTO technicians (name, email, active) VALUES (?, ?, ?)",
        (technician.name, technician.email, 1 if technician.active else 0)
    )

    conn.commit()

    technician_id = cursor.lastrowid

    conn.close()

    return Technician(
        id= technician_id,
        name=technician.name,
        email=technician.email,
        active=technician.active
    )
```

La función de este método es **persistir (guardar) un nuevo registro de técnico en la base de datos física y devolver al sistema un objeto estructurado que contenga el identificador único generado**.

- **def create(self, technician: Technician)**: declaración de la firma del método.
Recibe como argumento una instancia de la clase modelo Technician.
En el momento de la recepción, este objeto carece de identificador (su atributo id es None) dado que aún no existe en la base de datos.
- **conn = get_connection()**: establece la conexión con la BBDD invocando la función importada del módulo de infraestructura (**db.py**).
- **cursor = conn.cursor()**: instancia el objeto cursor que actúa como interfaz de control para **transmitir comandos SQL** hacia el motor de la base de datos.
- **cursor.execute(...)**: ejecución de la sentencia de inserción > con 2 elementos fundamentales:
 - **Estructura SQL ("INSERT INTO technicians... VALUES (?, ?, ?)")**: define la instrucción de inserción en las columnas name, email y active.
La utilización de marcadores de posición (?) garantiza la parametrización de la consulta, delegando el saneamiento de las variables al conector y bloqueando vulnerabilidades de inyección SQL.
 - **Inyección de datos (technician.name, technician.email, 1 if technician.active else 0)**: construcción de la tupla con los valores a insertar.
Es crítica la conversión 1 if technician.active else 0. Dado que SQLite carece de un tipo de dato booleano nativo, esta expresión transforma el valor lógico de Python (True/False) en el valor numérico (1/0) requerido por la restricción CHECK de la tabla.
- **conn.commit()**: confirmación de la transacción.
Fuerza al motor de base de datos a volcar los cambios temporalmente almacenados en memoria hacia el archivo físico en el disco duro. La omisión de esta línea resultaría en la pérdida del registro al finalizar el método.

- **technician_id = cursor.lastrowid**: recuperación de la **clave primaria**.
Extrae mediante el cursor el último identificador autogenerado por la propiedad AUTOINCREMENT de SQLite durante la instrucción INSERT inmediatamente anterior.
- **conn.close()**: procedimiento de finalización.
Cierra el canal de comunicación y libera los recursos del sistema asociados a la conexión.
- **return Technician(...)**: generación del objeto de salida.
El método instancia y retorna un nuevo **objeto** de la clase **Technician**.
Copia los valores originales de nombre, email y estado, integrando de manera definitiva el id real asignado por la base de datos, lo que sincroniza el estado en memoria con el estado físico.

Método **find_all**:

```
def find_all(self) -> List[Technician]:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM technicians")
    rows = cursor.fetchall()
    conn.close()
    return [self.map_row_to_technician(row) for row in rows]
```

La función de este método es ejecutar una operación de lectura masiva para **extraer todos los registros** existentes en la tabla de **técnicos** y **retornar una colección estructurada de objetos del modelo**.

- **def find_all(self) -> List[Technician]**: declaración de la firma del método.
El indicador de tipo de retorno (-> **List[Technician]**) establece formalmente que la salida de la función será una lista compuesta exclusivamente por instancias de la clase modelo Technician.
- **conn = get_connection()**: establecimiento de la conexión con el archivo de la BBDD.
- **cursor = conn.cursor()**: instanciación del objeto cursor, necesario para canalizar la instrucción de lectura hacia el motor relacional.
- **cursor.execute("SELECT * FROM technicians")**: transmisión de la sentencia SQL de extracción.
Al tratarse de una consulta general que carece de la cláusula de filtrado WHERE, esta instrucción ordena al motor recuperar la **totalidad de las columnas y filas** de la tabla technicians.
No requiere parámetros dinámicos ni marcadores de posición.
- **rows = cursor.fetchall()**: extracción de los resultados.
El método **fetchall()** del cursor captura el conjunto completo de datos devuelto por la instrucción SELECT anterior y lo almacena en la variable **rows** como una lista de registros en formato crudo.
- **conn.close()**: procedimiento de cierre del canal de comunicación.
Es imperativo destacar la **ausencia de la instrucción conn.commit() en este bloque temporal**; al ser una operación de solo lectura, **no existen alteraciones en el estado de la base de datos** que requieran confirmación de persistencia.
- **return [self.map_row_to_technician(row) for row in rows]**: transformación de datos y retorno.
Esta instrucción utiliza la estructura sintáctica de comprensión de listas (*list comprehension*) para optimizar el proceso de mapeo.
Itera secuencialmente sobre la colección de registros crudos (rows) y aplica a cada elemento el método de conversión map_row_to_technician.
El resultado final es la generación y **retorno** de una **nueva lista estructurada** compuesta exclusivamente por objetos **Technician**.

Método `find_by_id`:

```
def find_by_id(self, technician_id:int) -> Optional[Technician]:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT * FROM technicians WHERE id = ?",
        (technician_id,)
    )
    row = cursor.fetchone()
    if row is None:
        return None
    conn.close()
    return self.map_row_to_technician(row)
```

La función de este método es **localizar y extraer un único registro de la tabla de técnicos** basándose en su identificador único (**clave primaria**), **devolviendo un objeto estructurado** si el registro existe, o un valor nulo en caso de no encontrar coincidencias.

- **def find_by_id(self, technician_id:int) -> Optional[Technician]:** declaración de la firma del método.
Requiere como parámetro de entrada un número entero (**technician_id**).
El indicador de tipo de retorno (-> **Optional[Technician]**) establece formalmente que la función devolverá una instancia válida del modelo Technician, o bien el valor None si la búsqueda no produce resultados.
- **conn = get_connection():** establecimiento de la conexión con el archivo de la BBDD.
- **cursor = conn.cursor():** instanciación del objeto cursor, necesario para la ejecución de la sentencia SQL.
- **cursor.execute("SELECT * FROM technicians WHERE id = ?", (technician_id,)):** transmisión de la sentencia SQL de extracción filtrada.
 - **Estructura SQL:** utiliza la cláusula WHERE id = ? para limitar la búsqueda exclusivamente a la fila cuyo identificador coincida con el valor proporcionado.
 - **Parametrización:** el valor de filtrado se inyecta mediante una tupla (technician_id,).

La coma final dentro del paréntesis es un requisito estricto de la sintaxis de Python para definir una tupla de un solo elemento, garantizando así la seguridad contra inyecciones SQL por parte del conector.
- **row = cursor.fetchone():** extracción del resultado.
A diferencia del método fetchall(), **fetchone()** recupera únicamente el **primer registro** que satisface la consulta.
Dado que el filtrado se realiza sobre una clave primaria (id), la base de datos garantiza matemáticamente un máximo de un resultado.
Si no hay coincidencias, este método devuelve None.
- **if row is None: return None:** evaluación condicional de existencia.
Si la base de datos no contiene el identificador solicitado, la variable row almacena el valor None.
El bloque condicional intercepta este escenario y fuerza la finalización temprana del método, retornando None para evitar errores en la fase de mapeo posterior.
(Nota técnica sobre el código: esta salida temprana omite la ejecución de conn.close(), lo que en un entorno de producción generaría una fuga de recursos al dejar la conexión abierta).
- **conn.close():** procedimiento de finalización estándar que cierra el canal de comunicación con la base de datos si el registro fue encontrado exitosamente.
- **return self.map_row_to_technician(row):** transformación de datos y retorno.
Si se confirma la existencia de datos en la variable row, esta se transfiere como argumento al método de conversión interno. Dicho método procesa el registro crudo y la función retorna finalmente el **objeto Technician** completamente estructurado

Método `map_row_to_technician`:

```
def map_row_to_technician(self, row: Row) -> Technician:
    return Technician(
        id=row[0],
        name=row[1],
        email=row[2],
        active=bool(row[3])
    )
```

Actúa como una **función de mapeo** (implementación del patrón *Data Mapper*). Su función única es **interceptar los datos crudos provenientes de la base de datos y traducirlos a una entidad del dominio** (un objeto Python, una instancia de la clase modelo **Technician** en este caso), centralizando esta lógica para evitar redundancias en los diferentes métodos de consulta.

- **def map_row_to_technician(self, row: Row) -> Technician:** declaración de la firma del método.
Requiere como parámetro de entrada **row**, tipado estáticamente como **Row** (el formato de registro devuelto por la librería `sqlite3`).
El indicador de tipo de retorno establece formalmente que la salida será siempre una instancia válida de la clase modelo **Technician**.
- **return Technician(...):** invocación del constructor de la clase modelo.
El método crea y devuelve simultáneamente la nueva instancia, inyectando los datos de la fila directamente como argumentos del objeto.
- **id = row[0], name = row[1], email = row[2]:** extracción posicional de datos.
Dado que el cursor de **SQLite devuelve** las **filas** como **estructuras indexadas** (similares a listas o tuplas), el método accede secuencialmente a cada columna mediante su índice numérico. Asigna la posición 0 a la clave primaria (**id**), la 1 al nombre y la 2 al correo electrónico. Este orden depende estrictamente de la disposición de las columnas definidas en la creación de la tabla.
- **active = bool(row[3]):** conversión explícita de tipos (**Type Casting**).
Constituye el paso más crítico del mapeo. Extrae el valor de la cuarta columna (índice 3), el cual está almacenado en la estructura de **SQLite** como un número entero (0 o 1).
Al procesarlo mediante la función nativa **bool()**, se fuerza la transformación del valor numérico al tipo de dato lógico de Python (**False** o **True**).
Esto garantiza que el objeto resultante cumpla con el tipado estricto definido en **models.py**.

> Análisis de la clase ***TicketRepository***Método ***create***:

```
def create(self, ticket: Ticket):
    conn = get_connection()
    cursor = conn.cursor()

    cursor.execute(
        "INSERT INTO tickets (title, description, created_by, status, technician_id) VALUES (?, ?, ?, ?, ?)",
        (ticket.title, ticket.description, ticket.created_by, ticket.status.value, ticket.technician_id)
    )

    conn.commit()

    ticket_id = cursor.lastrowid

    conn.close()

    return Ticket(
        id = ticket_id,
        title=ticket.title,
        description=ticket.description,
        created_by=ticket.created_by,
        status=ticket.status,
        technician_id=ticket.technician_id
    )
```

Insertar un nuevo registro en la tabla Tickets en la base de datos física y retornar el objeto estructurado incorporando el identificador único autogenerado.

- **def create(self, ticket: Ticket):** declaración del método.
Recibe un objeto Ticket cuyo atributo id es None inicialmente.
- **conn = get_connection() / cursor = conn.cursor():** apertura del canal de comunicación y creación del cursor para la transacción.
- **cursor.execute("INSERT INTO tickets... VALUES (?, ?, ?, ?, ?)", (...)):** ejecución de la sentencia parametrizada.
 - Se inyectan las propiedades del objeto secuencialmente; utiliza cinco marcadores de posición (?, ?, ?, ?, ?) para título, descripción, creador, estado y técnico asignado.
 - Destaca la extracción **ticket.status.value**: convierte el tipo enumerado de Python (**TicketStatus.OPEN**) en su representación de texto ("**OPEN**") para ser compatible con la base de datos.
- **conn.commit():** confirmación de la transacción para persistir el nuevo registro en el disco.
- **ticket_id = cursor.lastrowid:** captura del valor de la clave primaria asignada por la restricción AUTOINCREMENT de SQLite durante la inserción.
- **conn.close():** cierre de la conexión y liberación de recursos.
- **return Ticket(...):** instanciación y retorno de un nuevo **objeto Ticket** que replica los datos originales pero integra el id definitivo.

Método `find_all`:

```
def find_all(self) -> List[Ticket]:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM tickets")
    rows = cursor.fetchall()
    conn.close()
    return [self.map_row_to_ticket(row) for row in rows]
```

Ejecuta una lectura masiva para *extraer todas las incidencias registradas, retornando una colección de objetos* del dominio (de la *clase Ticket*).

- **def find_all(self) -> List[Ticket]:** declaración con indicación de tipo que garantiza el **retorno** de una **lista** de **objetos Ticket**.
- **cursor.execute("SELECT * FROM tickets"):** ejecución de consulta global sin filtros, ordenando al motor recuperar **toda** la tabla.
- **rows = cursor.fetchall():** extracción y almacenamiento en memoria de todos los registros resultantes en formato crudo.
- **conn.close():** cierre del canal de comunicación.
No se requiere **commit()** al ser una operación de solo lectura.
- **return [self.map_row_to_ticket(row) for row in rows]:** utilización de comprensión de listas para iterar sobre los resultados crudos, aplicar la función de mapeo a cada uno y devolver la lista estructurada final de objetos *Ticket*.

Método `find_by_id`:

```
def find_by_id(self, ticket_id:int) -> Optional[Ticket]:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT * FROM tickets WHERE id = ?",
        (ticket_id,)
    )
    row = cursor.fetchone()
    if row is None:
        return None
    conn.close()
    return self.map_row_to_ticket(row)
```

Localiza y extraer una incidencia específica mediante su clave primaria, retornando el objeto correspondiente o valor nulo si no existe.

- **cursor.execute("SELECT * FROM tickets WHERE id = ?", (ticket_id,)):** consulta filtrada parametrizada.
El identificador se pasa como una tupla de un solo elemento para prevenir inyección SQL.
- **row = cursor.fetchone():** extracción del primer (y único) registro coincidente con la **clave primaria**.
- **if row is None: return None:** evaluación de existencia.
Interrumpe la ejecución y retorna nulo si el identificador no se encuentra en la base de datos (omitiendo nuevamente el `conn.close()` lo cual no debería ser correcto).
- **return self.map_row_to_ticket(row):** mapeo y retorno del objeto estructurado si la consulta fue exitosa.

Método `update_status`:

```
def update_status(self, ticket_id:int, status: TicketStatus) -> bool:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        "UPDATE tickets SET status = ? WHERE id = ?",
        (status.value, ticket_id)
    )
    conn.commit()
    update = cursor.rowcount > 0
    conn.close()
    return update
```

Modifica el estado de una incidencia existente, retornando un valor lógico que confirma el éxito o fracaso de la operación; modifica de manera independiente la **columna `status`** de un registro existente.

- **def update_status(self, ticket_id:int, status: TicketStatus) -> bool:** el método recibe el ID objetivo y el nuevo estado (tipo Enum).
Garantiza un retorno booleano.
- **cursor.execute("UPDATE tickets SET status = ? WHERE id = ?", (status.value, ticket_id)):** instrucción de modificación. Extrae el texto del enumerado (**status.value**) y lo inyecta junto al ID para localizar el registro exacto.
- **conn.commit():** confirmación indispensable para persistir la modificación en el archivo físico.
- **update = cursor.rowcount > 0:** evaluación del impacto de la consulta.
La propiedad **rowcount** devuelve el número de filas alteradas. Si es mayor que cero, la actualización fue efectiva; si es cero, el ID proporcionado no existía.
- **conn.close():** cierre de la conexión.
- **return update:** retorno del resultado de la evaluación booleana; True si se actualizó el registro, False si el ID no existía.

Método `assign_technician`:

```
def assign_technician(self, ticket_id: int, technician_id:int) -> bool:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        "UPDATE tickets SET technician_id = ?, status = ? WHERE id = ?",
        (technician_id, TicketStatus.IN_PROGRESS.value, ticket_id)
    )
    conn.commit()
    update = cursor.rowcount > 0
    conn.close()
    return update
```

Vincula un técnico a una incidencia y, simultáneamente, **fuerza la transición del estado del ticket a "En progreso"**.

- **cursor.execute("UPDATE tickets SET technician_id = ?, status = ? WHERE id = ?", (...)):** modificación múltiple en una única transacción.
 - Modifica dos columnas en una sola transacción, con **technician_id** y **TicketStatus.IN_PROGRESS.value**., ticket_id), inyectando como parámetros estos dos valores y **ticket_id**.
 - Se aplica una regla de negocio directamente en la capa de datos al inyectar estáticamente el valor "IN_PROGRESS", sobrescribiendo el estado anterior.
- **conn.commit() / update = cursor.rowcount > 0:** persistencia de los cambios y evaluación de filas afectadas para confirmar la operación.
- **return update:** retorno del estado de éxito de la transacción.

Método `find_open_tickets`:

```
def find_open_tickets(self) -> List[Ticket]:
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("""
        SELECT *
        FROM tickets
        WHERE status = ?
    """, (TicketStatus.OPEN.value, ))
    rows = cursor.fetchall()
    conn.close()

    return [self.map_row_to_ticket(row) for row in rows]
```

Recupera una colección filtrada que contenga exclusivamente las **incidencias que aún no han sido atendidas** (estado abierto).

- **cursor.execute("SELECT * FROM tickets WHERE status = ?", (TicketStatus.OPEN.value,))**: instrucción de lectura con filtro por columna.
Se inyecta el valor de texto correspondiente al **enumerado OPEN** como parámetro de búsqueda.
- **rows = cursor.fetchall()**: recupera todos los registros que satisfacen la condición del filtro.
- **return [self.map_row_to_ticket(row) for row in rows]**: mapeo iterativo de los resultados crudos a **objetos Ticket** mediante comprensión de listas.

Método `map_row_to_ticket`:

```
def map_row_to_ticket(self, row: Row) -> Ticket:
    print(row[0])
    return Ticket(
        id=row[0],
        title=row[1],
        description=row[2],
        created_by=row[3],
        status=TicketStatus(row[4]),
        technician_id=row[5]
    )
```

Actúa como la función de **mapeo (Data Mapper) exclusiva para la entidad Ticket**, traduciendo los registros crudos de **SQLite a objetos del dominio (tipo Ticket)**.

- **def map_row_to_ticket(self, row: Row) -> Ticket**: firma del método que recibe la fila nativa de base de datos.
- **print(row[0])**: instrucción de depuración (debugging) presente en el código original.
Imprime el ID en la consola estándar cada vez que se mapea un ticket. (En un entorno de producción, esta línea debería ser eliminada).
- **return Ticket(...)**: invocación al constructor inyectando valores posicionales (row[0] a row[5]).
- **status=TicketStatus(row[4])**: conversión inversa de tipos.
Toma la cadena de texto de la columna estado (ej. "OPEN") y la pasa como argumento al **enumerado TicketStatus**. Esto fuerza a Python a buscar la constante correspondiente y asignarla, garantizando que el objeto devuelto respete el tipado estricto.

3.3.4 `service.py`

Este archivo constituye la **Capa de Servicio**.

Su responsabilidad arquitectónica es actuar **como intermediario entre la interfaz de usuario** (o **controlador**) y la **capa de acceso a datos**.

Orquesta las **operaciones**, centraliza la **lógica de negocio**, aplica las **reglas de validación** y coordina las **llamadas** a los **repositorios** correspondientes.

```
from repository import TechnicianRepository, TicketRepository
from models import Ticket, TicketStatus, Technician
from typing import List, Optional
class Service:
    def __init__(self):
        self.technician_repository = TechnicianRepository()
        self.ticket_repository = TicketRepository()

    def register_technician(self, name: str, email: str) -> Technician:
        technician = Technician(
            id = None,
            name=name,
            email=email
        )
        return self.technician_repository.create(technician)

    def create_ticket(self, title: str, description: str, created_by:str) -> Ticket:
        ticket = Ticket(
            id = None,
            title=title,
            description=description,
            created_by=created_by,
            status=TicketStatus.OPEN,
            technician_id=None
        )

        return self.ticket_repository.create(ticket)

    def assign_ticket(self, ticket_id:int, technician_id:int) -> None:
        ticket = self.ticket_repository.find_by_id(ticket_id)
        if ticket is None:
            raise ValueError(f"El ticket {ticket_id} no existe")
        technician = self.technician_repository.find_by_id(technician_id)

        if technician is None:
            raise ValueError(f"El tecnico {technician_id} no existe")
        if not technician.active:
            raise ValueError("El tecnico no esta activo")
        if ticket.status == TicketStatus.CLOSED:
            raise ValueError("No se puede asignar un ticket cerrado")

        update = self.ticket_repository.assign_technician(ticket_id, technician_id)
        if not update:
            raise ValueError("No se ha podido asignar un ticket")

    def close_ticket(self, ticket_id: int) -> None:
        ticket = self.ticket_repository.find_by_id(ticket_id)
        if ticket is None:
            raise ValueError(f"El ticket {ticket_id} no existe")
        if ticket.status == TicketStatus.CLOSED:
            raise ValueError("El ticket ya se encuentra cerrado")

        update = self.ticket_repository.update_status(ticket_id, TicketStatus.CLOSED)
        if not update:
            raise ValueError("No se pudo cerrar el ticket")

    def get_all_tickets(self) -> List[Ticket]:
        return self.ticket_repository.find_all()

    def get_all_technician(self) -> List[Technician]:
        return self.technician_repository.find_all()

    def get_open_tickets(self) -> List[Ticket]:
        return self.ticket_repository.find_open_tickets()

    def get_ticket_by_id(self, ticket_id: int) -> Optional[Ticket]:
        return self.ticket_repository.find_by_id(ticket_id)
```

> Importaciones

La cabecera del archivo contiene las importaciones necesarias para establecer las dependencias estructurales y lógicas de la Capa de Servicio. Su desglose de ejecución es el siguiente:

- **from repository import TechnicianRepository, TicketRepository**
 - **Propósito:** establecer la **dependencia** con la **Capa de Acceso a Datos**.
 - **Mecanismo:** importa las clases encargadas de gestionar las **transacciones SQL**. Esto permite que el constructor de Service pueda **instanciar ambos repositorios y delegar en ellos las operaciones de lectura y escritura**, garantizando que la capa de negocio permanezca agnóstica respecto a la infraestructura de la base de datos.
- **from models import Ticket, TicketStatus, Technician**
 - **Propósito:** integrar las **entidades del dominio** para el procesamiento de la lógica de negocio.
 - **Mecanismo:** la capa de servicio requiere estas clases estructurales por dos motivos fundamentales:
 1. **Instanciar nuevos objetos** (ej. Ticket(...) o Technician(...)) al procesar solicitudes de creación antes de enviarlas al repositorio.
 2. **Operar con las propiedades de los objetos devueltos por la base de datos** y utilizar el enumerado TicketStatus para evaluar condiciones estrictas (ej. if ticket.status == TicketStatus.CLOSED).
- **from typing import List, Optional**
 - **Propósito:** habilitar el **tipado** estricto en las firmas de los métodos.
 - **Mecanismo:** importa los módulos de tipado para **documentar y restringir los retornos de las funciones**. **List** se utiliza en los métodos de consulta masiva (ej. devolver un conjunto de tickets). **Optional** se aplica en búsquedas específicas donde el resultado puede ser un objeto o un valor nulo (ej. búsqueda por identificador).

> Constructor de la clase: `__init__(self)`

Propósito general: *inicializar las dependencias necesarias para que el servicio pueda interactuar con la base de datos a través de la capa de acceso a datos.*

- **self.technician_repository = TechnicianRepository()**: instancia el **repositorio de técnicos** y lo asigna como un atributo de la clase.
- **self.ticket_repository = TicketRepository()**: instancia el **repositorio de tickets** y lo asigna como un atributo.

Nota técnica: A partir de este momento, *cualquier método del servicio utilizará estas dos instancias para delegar las operaciones de lectura y escritura*, garantizando que **el servicio nunca ejecute sentencias SQL directamente**.

> Métodos:

> Métodos de creación

- **register_technician(self, name: str, email: str) -> Technician** > procesa el alta de un nuevo técnico en el sistema.
- **create_ticket(self, title: str, description: str, created_by: str) -> Ticket** > inicia el ciclo de vida de una nueva incidencia.

> Métodos de operación y validación de reglas de negocio

- **assign_ticket(self, ticket_id: int, technician_id: int) -> None** > vincula un técnico a una incidencia garantizando la integridad de las reglas de negocio del sistema.
- **close_ticket(self, ticket_id: int) -> None** > finalizar el ciclo de vida de una incidencia (pasa el estado a cerrado).

> Métodos de consulta (Delegación directa)

Los siguientes cuatro métodos actúan exclusivamente como **interfaces de paso (passthrough methods)**.

No contienen lógica condicional ni transformaciones; su función es **recibir la petición del nivel superior y devolver directamente la respuesta del repositorio correspondiente**:

- **get_all_tickets(self) -> List[Ticket]**: invoca y retorna self.ticket_repository.find_all().
- **get_all_technician(self) -> List[Technician]**: invoca y retorna self.technician_repository.find_all().
- **get_open_tickets(self) -> List[Ticket]**: invoca y retorna self.ticket_repository.find_open_tickets().
- **get_ticket_by_id(self, ticket_id: int) -> Optional[Ticket]**: invoca y retorna self.ticket_repository.find_by_id(ticket_id).

A continuación, se desglosan uno a uno.

> Métodos de creación

Método **register_technician**:

```
def register_technician(self, name: str, email: str) -> Technician:
    technician = Technician(
        id = None,
        name=name,
        email=email
    )
    return self.technician_repository.create(technician)
```

Procesa la solicitud de alta de un nuevo técnico en el sistema, construyendo la **entidad** correspondiente en memoria y delegando su **almacenamiento** físico a la capa de acceso a datos.

- **def register_technician(self, name: str, email: str) -> Technician**: declaración de la firma del método. Define dos parámetros de entrada obligatorios tipados como cadenas de texto (**name** y **email**). El indicador de tipo de retorno documenta estrictamente que la ejecución culminará con la **entrega de un objeto validado de la clase Technician**.
- **technician = Technician(id = None, name=name, email=email)**: instanciación del modelo de dominio. El método construye un nuevo objeto en memoria invocando al constructor de la **clase Technician**.
 - **id = None**: se asigna un valor nulo de forma explícita. En esta fase del ciclo de vida del objeto, **el registro aún no existe en la base de datos**, por lo que carece de identificador. La **generación de la clave primaria recae en la infraestructura de SQLite**.
 - **Asignación implícita**: aunque no se especifica en la llamada, el **atributo active** de este objeto adquiere automáticamente el valor booleano **True**, aplicando la regla de negocio definida por defecto en la estructura de la clase dentro de **models.py**.
- **return self.technician_repository.create(technician)**: delegación de persistencia y retorno. El servicio transfiere el objeto recién creado en memoria al método **create** del **repositorio** asociado (**self.technician_repository**). Dicho **repositorio** se encarga de ejecutar la **instrucción SQL** de **inserción**, recuperar el identificador generado y devolver el objeto completamente estructurado. La instrucción **return** propaga este objeto final de vuelta a la capa de presentación que invocó la función original.

Método `create_ticket`:

```
def create_ticket(self, title: str, description: str, created_by:str) -> Ticket:
    ticket = Ticket(
        id = None,
        title=title,
        description=description,
        created_by=created_by,
        status=TicketStatus.OPEN,
        technician_id=None
    )

    return self.ticket_repository.create(ticket)
```

Inicia el ciclo de vida de una nueva incidencia en el sistema, *construyendo la entidad con sus valores predeterminados de estado* y *delegando su almacenamiento físico* a la capa de acceso a datos.

- **def create_ticket(self, title: str, description: str, created_by:str) -> Ticket:** declaración de la firma del método.
Define 3 parámetros de entrada obligatorios de tipo texto (**title**, **description** y **created_by**).
El indicador de tipo de retorno documenta que la función entregará un objeto validado de la **clase Ticket**.
- **ticket = Ticket(...):** instanciación del modelo de dominio.
El método construye un nuevo objeto en memoria invocando al **constructor** de la **clase Ticket**.
Se inyectan los parámetros recibidos junto con valores estáticos que responden a reglas de negocio fundamentales:
 - **id = None:** se asigna explícitamente un valor nulo, ya que la entidad aún no ha sido registrada en la base de datos y carece de clave primaria.
 - **title=title, description=description, created_by=created_by:** asignación directa de los datos proporcionados en la invocación del método.
 - **status=TicketStatus.OPEN:** aplicación de regla de negocio inicial.
Fuerza a **que toda incidencia recién registrada** adquiera de forma inamovible el estado **abierto**.
 - **technician_id=None:** aplicación de regla de negocio de asignación.
Establece la ausencia de un técnico responsable en el momento de creación, dejando el campo opcional vacío.
- **return self.ticket_repository.create(ticket):** delegación de persistencia y retorno.
El servicio transfiere el objeto construido en memoria al método **create** del **repositorio** asociado (**self.ticket_repository**). Este repositorio ejecuta la **instrucción SQL** de **inserción**, recupera el identificador único generado por SQLite y devuelve el objeto final.
La instrucción **return** propaga este objeto de vuelta a la capa superior.

Carencia detectada según requisitos: este método omite actualmente las validaciones exigidas en la especificación del proyecto relativas a la **longitud del título** y el formato del **correo electrónico**.

> Métodos de operación y validación de las reglas de negocio

Método `assign_ticket`:

```
def assign_ticket(self, ticket_id:int, technician_id:int) -> None:
    ticket = self.ticket_repository.find_by_id(ticket_id)
    if ticket is None:
        raise ValueError(f"El ticket {ticket_id} no existe")
    technician = self.technician_repository.find_by_id(technician_id)

    if technician is None:
        raise ValueError(f"El tecnico {technician_id} no existe")
    if not technician.active:
        raise ValueError("El tecnico no esta activo")
    if ticket.status == TicketStatus.CLOSED:
        raise ValueError("No se puede asignar un ticket cerrado")

    update = self.ticket_repository.assign_technician(ticket_id, technician_id)
    if not update:
        raise ValueError("No se ha podido asignar un ticket")
```

Vincular un técnico específico a una incidencia existente, validando rigurosamente que la operación cumpla con todas las reglas de negocio establecidas antes de delegar la actualización del estado a la base de datos.

- **def assign_ticket(self, ticket_id:int, technician_id:int) -> None:** declaración de la firma del método. Requiere los **identificadores** únicos del **ticket** y del **técnico** involucrados. El indicador de tipo de retorno (-> None) establece que la función ejecuta una acción modificadora pero **no devuelve ningún objeto** si tiene éxito (aunque puede lanzar excepciones si falla).
- **ticket = self.ticket_repository.find_by_id(ticket_id):** el servicio solicita el ticket a la capa de datos.
 - **if ticket is None: raise ValueError(...):** validación de existencia de la incidencia. Si el repositorio retorna nulo, se interrumpe inmediatamente el flujo de ejecución lanzando una **excepción ValueError, impidiendo operar sobre un registro inexistente**.
- **technician = self.technician_repository.find_by_id(technician_id) :** el servicio solicita el técnico a la capa de datos.
 - **if technician is None: raise ValueError(...):** validación de existencia del técnico. Sigue el mismo patrón de salvaguarda que el paso anterior.
 - > Esta condición satisface la regla de negocio explícita: "No se puede asignar un ticket a un técnico inexistente".
- **if not technician.active: raise ValueError(...)** validación de **estado operativo del técnico**. Evalúa el atributo active del objeto Technician cargado en memoria. Si el técnico está marcado como inactivo (False), se aborta la asignación.
 - > Esta condición satisface la regla de negocio explícita: "No se puede asignar un ticket a un técnico inactivo".
- **if ticket.status == TicketStatus.CLOSED: raise ValueError(...):** validación del ciclo de vida de la incidencia. Comprueba el **estado** actual del objeto **Ticket**. Se **impide la reasignación de recursos** a incidencias que ya han sido dadas de baja temporal o definitivamente.
- **update = self.ticket_repository.assign_technician(ticket_id, technician_id):** delegación de persistencia. Una vez superados todos los filtros de validación, el **servicio invoca el método de actualización del repositorio**. Dicho método **ejecuta la transacción SQL** y retorna un valor lógico (True/False) indicando el éxito de la modificación.
- **if not update: raise ValueError(...):** verificación final de integridad transaccional. Evalúa la respuesta booleana del repositorio.

Si el conector de base de datos no pudo alterar ninguna fila (por ejemplo, por una condición de carrera o un error de bloqueo en SQLite), se lanza una **excepción** final para notificar a la capa superior que la **operación**, aunque lógicamente válida, **no pudo materializarse físicamente**.

Método `close_ticket`:

```
def close_ticket(self, ticket_id: int) -> None:
    ticket = self.ticket_repository.find_by_id(ticket_id)
    if ticket is None:
        raise ValueError(f"El ticket {ticket_id} no existe")
    if ticket.status == TicketStatus.CLOSED:
        raise ValueError("El ticket ya se encuentra cerrado")

    update = self.ticket_repository.update_status(ticket_id, TicketStatus.CLOSED)
    if not update:
        raise ValueError("No se pudo cerrar el ticket")
```

Finaliza el ciclo de vida de una incidencia alterando su estado a cerrado, previa validación de su existencia y de su estado actual para garantizar la coherencia lógica de los datos.

- **def close_ticket(self, ticket_id: int) -> None:** declaración de la firma del método.
Requiere como parámetro el identificador único del **ticket**.
El indicador de tipo de **retorno** establece que la función es un procedimiento modificador que **no devuelve datos** lógicos al finalizar con éxito.
- **ticket = self.ticket_repository.find_by_id(ticket_id):** extracción de la entidad.
Delega en la capa de acceso a datos la búsqueda del registro actual del ticket para cargar su estado en memoria.
- **if ticket is None: raise ValueError(...):** validación de existencia.
Si el **repositorio** devuelve un **valor nulo**, se interrumpe la ejecución mediante una **excepción**, bloqueando cualquier intento de actualización sobre un registro inexistente.
- **if ticket.status == TicketStatus.CLOSED: raise ValueError(...):** validación de **redundancia de estado**.
Evalúa la propiedad **status** del objeto recuperado.
Si la incidencia ya figura como finalizada, se aborta la operación.
 > **Condición satisface la regla de negocio explícita** que dictamina que no se puede cerrar un ticket ya cerrado.
- **update = self.ticket_repository.update_status(ticket_id, TicketStatus.CLOSED):** delegación de persistencia.
Superadas las validaciones actuales, el servicio ordena a la capa de datos que ejecute la instrucción de actualización (**UPDATE**), inyectando de forma estática el nuevo estado **TicketStatus.CLOSED**.
El repositorio retorna un valor lógico sobre el éxito de la transacción.
- **if not update: raise ValueError(...):** verificación de **integridad transaccional**.
Evalúa la respuesta del repositorio.
Si el motor de la base de datos **no** confirmó la **alteración de ninguna fila**, se lanza una **excepción** final notificando el fracaso físico de la operación.

Carencia detectada según requisitos: omite la regla de negocio de nivel medio que prohíbe el cierre de un ticket si este no cuenta con un técnico asignado previamente (su `technician_id` es nulo).

> Métodos de consulta

Estos cuatro métodos actúan exclusivamente como interfaces de paso (*passthrough interfaces*) o delegadores. Su función arquitectónica es **encapsular el acceso a los repositorios**, permitiendo que las capas superiores (como `app.py`) puedan **solicitar información a la base de datos sin necesidad de interactuar directamente con la capa de datos**. Al tratarse de operaciones de **solo lectura**, no aplican reglas de negocio adicionales ni modificaciones de estado.

Método `get_all_tickets`:

```
def get_all_tickets(self) -> List[Ticket]:  
    return self.ticket_repository.find_all()
```

- **Firma del método:** no requiere parámetros de entrada adicionales al contexto de la clase (`self`). El **tipado** estricto garantiza el **retorno** de una **lista** de instancias de ***Ticket***.
- **Delegación y retorno:** `return self.ticket_repository.find_all()`. Transfiere la **petición** al **repositorio** de ***incidencias***, el cual ejecuta la **consulta SQL** global y devuelve la colección completa mapeada, que este método propaga hacia arriba de forma intacta.

Método `get_all_technician`:

```
def get_all_technician(self) -> List[Technician]:  
    return self.technician_repository.find_all()
```

- **Firma del método:** garantiza el **retorno** de una **lista** compuesta exclusivamente por instancias de la clase ***Technician***.
- **Delegación y retorno:** `return self.technician_repository.find_all()`. Transfiere la **solicitud** de lectura al **repositorio** de ***técnicos***. Es la interfaz estándar para **listar el personal registrado** en el sistema.

Método `get_open_tickets`:

```
def get_open_tickets(self) -> List[Ticket]:  
    return self.ticket_repository.find_open_tickets()
```

- **Firma del método:** define la salida como una **lista** de objetos ***Ticket***.
- **Delegación y retorno:** `return self.ticket_repository.find_open_tickets()`. Invoca el **método** especializado del **repositorio** que contiene el **filtro** por estado. Encapsula la lógica de **búsqueda** de incidencias pendientes, entregando únicamente aquellas cuyo atributo interno coincide con ***TicketStatus.OPEN***.

Método `get_ticket_by_id`:

```
def get_ticket_by_id(self, ticket_id: int) -> Optional[Ticket]:  
    return self.ticket_repository.find_by_id(ticket_id)
```

- **Firma del método:** requiere como **parámetro** el identificador único (***ticket_id***) tipado como entero. El indicador ***Optional[Ticket]*** documenta formalmente que el **resultado** puede ser una **entidad válida** o un valor **nulo** si no hay coincidencias.
- **Delegación y retorno:** `return self.ticket_repository.find_by_id(ticket_id)`. Transfiere el identificador objetivo al repositorio para ejecutar la consulta filtrada por clave primaria, **retornando la entidad mapeada** correspondiente o ***None*** en caso de ausencia.

3.3.5 `app.py`

> **Análisis de funcionalidad**

Este archivo constituye el **Punto de Entrada**.

El archivo `app.py` no constituye una interfaz de usuario final ni un controlador de enrutamiento completo (como los que se implementarían con un *framework* web).

Actúa exclusivamente como un **cliente de prueba o punto de entrada (Entry Point) provisional**.

Su función es instanciar la capa de servicio e **invocar secuencialmente algunos de sus métodos** para verificar que la arquitectura y la conexión a la base de datos operan correctamente

> **Importación de dependencias**

- **`from service import Service`**: importa la clase principal de la capa de lógica de negocio. Es fundamental notar la **ausencia** de **importaciones** relativas a **`repository.py` o `db.py`**. Esto confirma el cumplimiento estricto de la arquitectura en capas: el cliente (**`app.py`**) ignora por completo cómo se almacenan los datos y se comunica únicamente con el "cerebro" de la aplicación (**`Service`**).

> **Función de utilidad visual**

- **`def print_separator() -> None::`**
`print("\n" + "=" * 60 + "\n")`

Método auxiliar diseñado para mejorar la legibilidad en la consola estándar durante las pruebas.

Su ejecución imprime un **salto de línea** y una **línea divisoria** construida mediante la multiplicación de caracteres (`"=" * 60`). No aporta valor lógico a la arquitectura del sistema.

> **Función principal de ejecución**

- **`def main() -> None::`**
`service = Service()`
`...`

Encapsula el **bloque de pruebas**. Su ejecución sigue un flujo lineal:

- **`service = Service()`**: instancia el servicio.

Al hacerlo, el constructor de `Service` (analizado previamente) **inicializa automáticamente los repositorios de técnicos y tickets**, dejando la aplicación lista para operar.

> **Bloque de control de ejecución**

- **`if __name__ == "__main__":`** estructura de control estándar en Python.
`main()`

Evalúa si el archivo está siendo ejecutado directamente por el intérprete de Python (ej. ejecutando `python app.py` en la terminal):

- > Si es el caso, la variable de entorno `__name__` adquiere el valor `"__main__"` y se invoca la función **`main()`**.
- > Si este archivo fuera importado desde otro módulo, esta condición bloquearía la ejecución automática de las pruebas.

> **Comprobaciones realizadas por el profesor (app.py)**

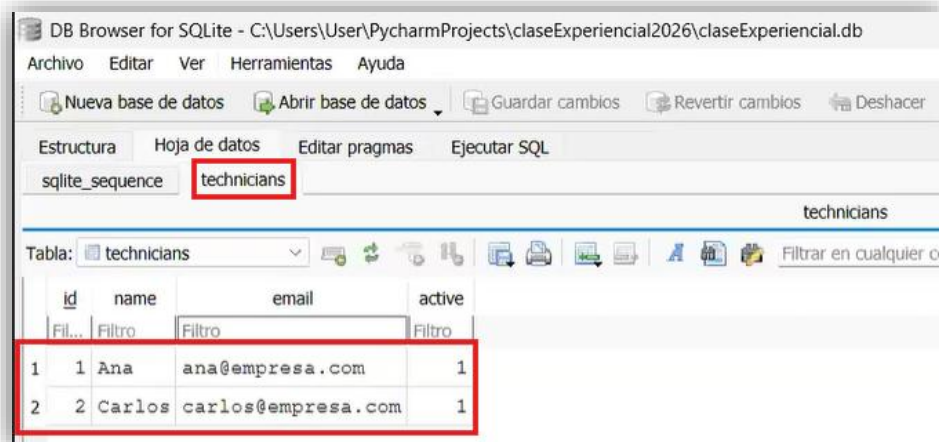
Con el fin de comprobar la funcionalidad del proyecto, se van realizando una serie de comprobaciones escribiendo y modificando el código del archivo "app.py" sobre la marcha. A continuación, se muestran las operaciones mostradas en clase.



```
1 from service import Service
2
3 def print_separator() -> None: 1 usage
4     print("\n" + "=" * 60 + "\n")
5
6 def main()->None: 1 usage
7     service = Service()
8
9     print_separator()
10    print("1. Registrando técnicos...")
11    tech1 = service.register_technician(name: "Ana", email: "ana@empresa.com")
12    tech2 = service.register_technician(name: "Carlos", email: "carlos@empresa.com")
13
14
15 if __name__ == "__main__":
16     main()
```



```
=====
1. Registrando tecnicos...
Process finished with exit code 0
```



DB Browser for SQLite - C:\Users\User\PycharmProjects\claseExperiencial2026\claseExperiencial.db

Archivo Editar Ver Herramientas Ayuda

Nueva base de datos Abrir base de datos Guardar cambios Revertir cambios Deshacer

Estructura Hoja de datos Editar pragmas Ejecutar SQL

sqlite_sequence technicians

Tabla: technicians

	id	name	email	active
	Fi...	Filtro	Filtro	Filtro
1	1	Ana	ana@empresa.com	1
2	2	Carlos	carlos@empresa.com	1

```
def main()->None: 1 usage
    service = Service()

    print_separator()
    print("2. Creando tickets...")
    ticket1 = service.create_ticket(
        title: "VPN no funciona",
        description: "El empleado no pudo conectarse",
        created_by: "Juan"
    )
    ticket2 = service.create_ticket(
        title: "Portátil muy lento",
        description: "El equipo tarda",
        created_by: "María"
    )

if __name__ == "__main__":
    main()
```

```
=====

2. Creando tickets...

Process finished with exit code 0
```

DB Browser for SQLite - C:\Users\User\PycharmProjects\claseExperiencial2026\claseExperiencial.db

Archivo Editar Ver Herramientas Ayuda

Nueva base de datos Abrir base de datos Guardar cambios Revertir cambios Des

Estructura Hoja de datos Editar pragmas Ejecutar SQL

sqlite_sequence technicians tickets tickets

Tabla: tickets

	id	title	description	created_by	status	technician_id
Fil...	Filtro	Filtro	Filtro	Filtro	Filtro	Filtro
1	1	VP...	El ...	juan	OPEN	NULL
2	2	Po...	El ...	Maria	OPEN	NULL

```
def main()->None: 1 usage
    service = Service()
    print("1. Registrando técnico")
    tech3 = service.register_technician(name="Antonio", email="antonio@empresa.com")

    print_separator()
    print("2. Creando tickets...")
    ticket3 = service.create_ticket(
        title="VPN no funciona",
        description="El empleado no pudo conectarse",
        created_by="Antonio"
    )
    ticket2 = service.create_ticket(
        title="Portátil muy lento",
        description="El equipo tarda",
        created_by="Maria"
    )
    print_separator()
    print("3. Asignando el ticket 3 al técnico 3...")
    service.assign_ticket(ticket3.id, tech3.id)

    print_separator()
    print("4. Cerrando el ticket 3...")
    service.close_ticket(ticket3.id)

if __name__ == "__main__":
    main()
```

DB Browser for SQLite - C:\Users\User\PycharmProjects\claseExperiencial2026\claseExperiencial.db

Archivo Editar Ver Herramientas Ayuda

Nueva base de datos Abrir base de datos Guardar cambios Revertir cambios Deshacer Abrir proyecto

Estructura Hoja de datos Editar pragmas Ejecutar SQL

sqlite_sequence tickets

tickets

Tabla: tickets

Filtrar en cualquier columna

	id	title	description	created_by	status	technician_id
	Fil...	Filtro	Filtro	Filtro	Filtro	Filtro
1	1	VPN no funciona	El empleado no pudo conectarse	juan	CLOSED	1
2	2	Portatil muy lento	El equipo tarda	Maria	OPEN	NULL
3	3	VPN no funciona	El empleado no pudo conectarse	antonio	OPEN	NULL

> **Propuesta de operaciones / comprobaciones (app2.py)**

El archivo "app.py" facilitado por el profesor contiene algunas pruebas realizadas sobre la marcha y es incompleto. Para poder estudiar la app al completo se genera un **código de prueba** en un archivo que denominamos "**app2.py**" en el que se tratan cronológicamente todas las opciones para las que está preparado el programa: creación de técnicos, creación de tickets, asignación a técnicos, estado de los tickets.... pudiendo así comprobar el funcionamiento de los distintos métodos.

El código incluye sentencias que **pausan el flujo** después de cada acción (hasta pulsar "**Enter**" u otra tecla) para poder **auditar cada acción y su efecto en la base de datos paso a paso**.

Estructura propuesta para app2.py

```
import os
from service import Service
from db import init_db

def pause_and_prompt() -> None:
    """
    Interrumpe el flujo de ejecución temporalmente.
    Permite al usuario auditar la base de datos en SQLite antes de continuar.
    """
    input("\n[Pausa] Operación finalizada. Presione [ENTER] para continuar al siguiente paso...")
    print("\n" + "=" * 80 + "\n")

def print_step_header(step_number: int, description: str) -> None:
    """Imprime un encabezado formal para cada acción secuencial."""
    print(f"--- PASO {step_number}: {description} ---")

def main() -> None:
    # -----
    # PASO 1: Inicialización de la Infraestructura
    # -----
    print_step_header(1, "Inicialización de la base de datos (Creación de tablas)")
    if os.path.exists("claseExperiencial.db"):
        print("Aviso: El archivo 'claseExperiencial.db' ya existe.")
        print("Para evitar errores de creación de tablas, asegúrese de haberlo borrado antes de esta prueba.")
    else:
        init_db()
        print("> Tablas 'technicians' y 'tickets' creadas exitosamente en el esquema relacional.")

    pause_and_prompt()

    # Instanciación de la capa de Lógica de Negocio
    service = Service()

    # -----
    # PASO 2: Creación de Entidades (Técnicos)
    # -----
    print_step_header(2, "Registro de técnicos operativos en el sistema")
    tech1 = service.register_technician("Ana García", "ana@techsolutions.com")
    tech2 = service.register_technician("Luis Pérez", "luis@techsolutions.com")
    print(f"> Entidad persistida: {tech1}")
    print(f"> Entidad persistida: {tech2}")

    pause_and_prompt()

    # -----
    # PASO 3: Lectura General (Técnicos)
    # -----
    print_step_header(3, "Consulta del listado completo de técnicos")
    all_techs = service.get_all_technician()
    for t in all_techs:
        print(f" - {t}")

    pause_and_prompt()
```

```
# -----
# PASO 4: Creación de Entidades (Tickets)
# -----
print_step_header(4, "Registro de nuevas incidencias (Estado inicial: OPEN)")
ticket1 = service.create_ticket("Fallo en red", "Pérdida de paquetes en planta baja", "empleado1@empresa.com")
ticket2 = service.create_ticket("Hardware dañado", "Teclado sin respuesta", "empleado2@empresa.com")
print(f"> Entidad persistida: {ticket1}")
print(f"> Entidad persistida: {ticket2}")

pause_and_prompt()

# -----
# PASO 5: Lectura Filtrada (Tickets Abiertos)
# -----
print_step_header(5, "Consulta de incidencias pendientes (Tickets ABIERTOS)")
open_tickets = service.get_open_tickets()
for t in open_tickets:
    print(f" - {t}")

pause_and_prompt()

# -----
# PASO 6: Modificación / Regla de Negocio (Asignación)
# -----
print_step_header(6, "Asignación de un técnico a una incidencia")
print(f"> Asignando el Ticket ID {ticket1.id} al Técnico ID {tech1.id}...")
service.assign_ticket(ticket1.id, tech1.id)
print("> Operación exitosa. El estado en base de datos debe ser ahora IN_PROGRESS y contener la clave foránea.")

pause_and_prompt()

# -----
# PASO 7: Búsqueda por Identificador
# -----
print_step_header(7, "Recuperación de entidad específica por ID")
updated_ticket = service.get_ticket_by_id(ticket1.id)
print(f"> Estado actual de la entidad modificada: {updated_ticket}")

pause_and_prompt()

# -----
# PASO 8: Modificación / Regla de Negocio (Cierre)
# -----
print_step_header(8, "Cierre de incidencia completada")
print(f"> Procesando cierre para el Ticket ID {ticket1.id}...")
service.close_ticket(ticket1.id)
print("> Operación exitosa. El estado en base de datos debe ser ahora CLOSED.")

pause_and_prompt()

# -----
# PASO 9: Lectura General y Verificación Final
# -----
print_step_header(9, "Consulta del listado completo de incidencias (Auditoría final)")
final_tickets = service.get_all_tickets()
for t in final_tickets:
    print(f" - {t}")

print("\n>>> Secuencia de pruebas completada. <<<")

if __name__ == "__main__":
    main()
```

> Importaciones

Las tres declaraciones de importación al inicio del archivo configuran las **dependencias** necesarias para que el script de pruebas pueda **interactuar** con el **sistema operativo** y con las distintas **capas** de la **aplicación**.

> import os

- **Qué hace:** importa el módulo nativo de Python encargado de **proveer interfaces estandarizadas** para **interactuar** con el **sistema operativo** subyacente.
- **Por qué se incluye:** para utilizar la función **os.path.exists("claseExperiencial.db")**. Permite al código inspeccionar el disco duro y verificar si el archivo de SQLite ya existe en la ruta actual. Actúa como un mecanismo de control preventivo para evitar ejecutar la creación de tablas a ciegas.

SQLite tiene un comportamiento automático. Cuando en el archivo **db.py** se ejecuta la instrucción **sqlite3.connect("claseExperiencial.db")**, el conector busca el archivo. **Si no existe, lo crea automáticamente** en ese mismo instante. No se requiere ninguna instrucción previa para generar el archivo vacío.

Aunque el archivo exista o se acabe de crear automáticamente, por defecto está completamente vacío. No tiene tablas, ni columnas, ni reglas. Para crear esta estructura interna, se utiliza la función **init_db()** que contiene las sentencias SQL **CREATE TABLE**.

El código original de **db.py** utiliza simplemente **CREATE TABLE technicians**; SQLite intentará crear la tabla a la fuerza. Si el archivo .db ya existía de una prueba anterior y ya contenía las tablas, SQLite lanzará un error fatal: **sqlite3.OperationalError: table technicians already exists**, y el programa colapsará.

Para evitar el problema se presentan 2 posibles soluciones:

> Solución preferible: usar en el **db.py** la orden **CREATE TABLE IF NOT EXISTS...**: la propia base de datos gestionaría todo. Si la tabla ya existe, la ignora; si no está, la crea.

```
CREATE TABLE IF NOT EXISTS technicians (...)  
CREATE TABLE IF NOT EXISTS tickets (...)
```

> Mantener **db.py** igual y utilizar el **import os** y el **if os.path.exists(...)**: al comprobar con **os.path.exists("claseExperiencial.db")** si el archivo ya estaba en la carpeta del proyecto, el script decide si es seguro llamar a **init_db()** o si es mejor no llamarlo para evitar que el programa colapsara por intentar recrear tablas ya existentes.

> from service import Service

- **Qué hace:** importa exclusivamente la **clase Service** del módulo local **service.py** (la Capa de Lógica de Negocio).
- **Por qué se incluye:** es el **componente central para el funcionamiento del cliente**. Dado que **app2.py** actúa como la capa de presentación (o simulador de la misma), necesita instanciar esta clase (**service = Service()**) para **acceder a todos los métodos de operación y consulta**. Su inclusión respeta el diseño de la **arquitectura en capas**: permite a **app2.py** manipular el sistema sin tener que importar ni conocer los repositorios o las sentencias SQL.

> from db import init_db

- **Qué hace:** extrae la función específica **init_db** del archivo **db.py** (la Capa de Infraestructura), omitiendo el resto del contenido de dicho archivo (como **get_connection**).
- **Por qué se incluye:** normalmente, una aplicación en producción no crea sus tablas cada vez que se ejecuta el punto de entrada. Sin embargo, para satisfacer el requerimiento de este script de pruebas (ejecutar cronológicamente y documentar paso a paso desde el inicio), es necesario invocar la instrucción de configuración física. Esta importación provee el comando exacto para **construir el esquema relacional en SQLite** justo antes de comenzar a registrar técnicos e incidencias.

> Descripción de operaciones paso a paso

```
if __name__ == "__main__":
    main()
```

> Al correr **app2.py** > ejecuta **main()**> Ejecuta el **Paso 1**:

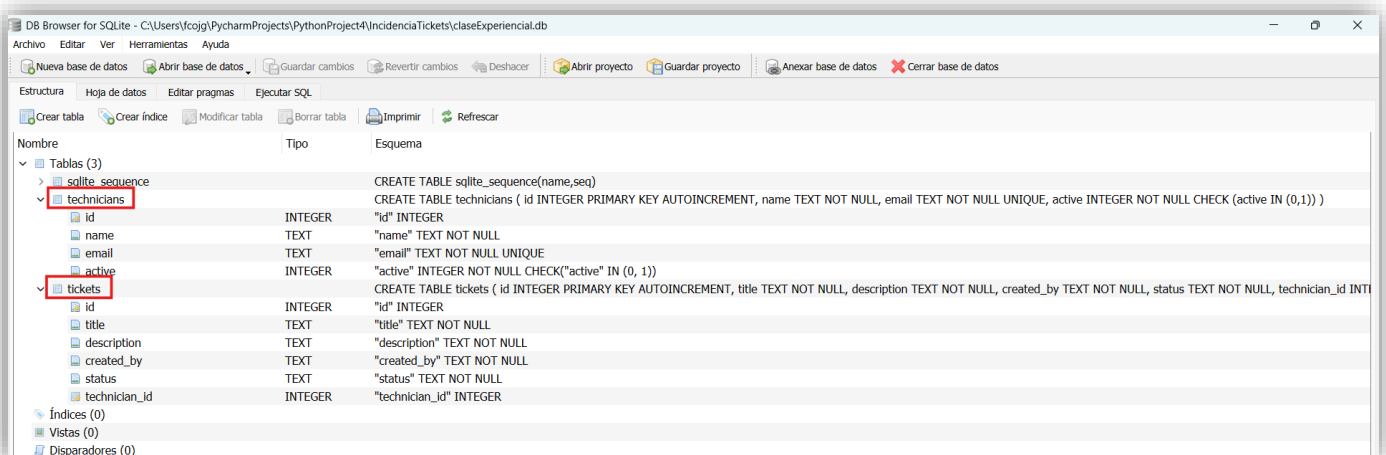
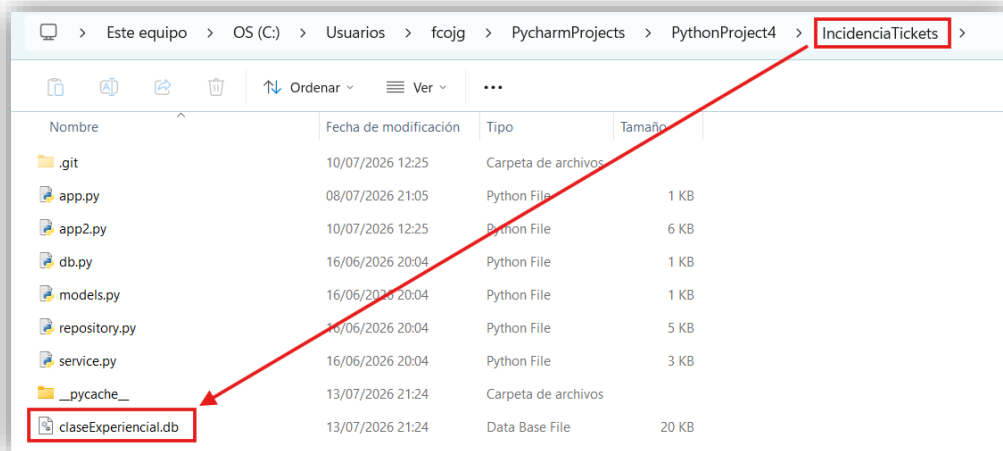
```
# -----
# PASO 1: Inicialización de la Infraestructura
# -----
print_step_header(1, "Inicialización de la base de datos (Creación de tablas)")
if os.path.exists("claseExperiencial.db"):
    print("Aviso: El archivo 'claseExperiencial.db' ya existe.")
    print("Para evitar errores de creación de tablas, asegúrese de haberlo borrado antes de esta prueba.")
else:
    init_db()
    print("> Tablas 'technicians' y 'tickets' creadas exitosamente en el esquema relacional.")

pause_and_prompt()
```

> Al no existir **"claseExperiencial.db"** > llama a **init_db()** importada de la capa de infraestructura (**db.py**), creando el archivo y las tablas:

```
C:\Users\fcojg\PycharmProjects\PythonProject4\.venv\Scripts\python.exe C:\Users\fcojg\PycharmProjects\PythonProject4\IncidenciaTickets\app2.py
--- PASO 1: Inicialización de la base de datos (Creación de tablas) ---
> Tablas 'technicians' y 'tickets' creadas exitosamente en el esquema relacional.

[Pausa] Operación finalizada. Presione [ENTER] para continuar al siguiente paso...
```



> Una vez creada la BBDD con las tablas se **instancia la capa de negocio**:

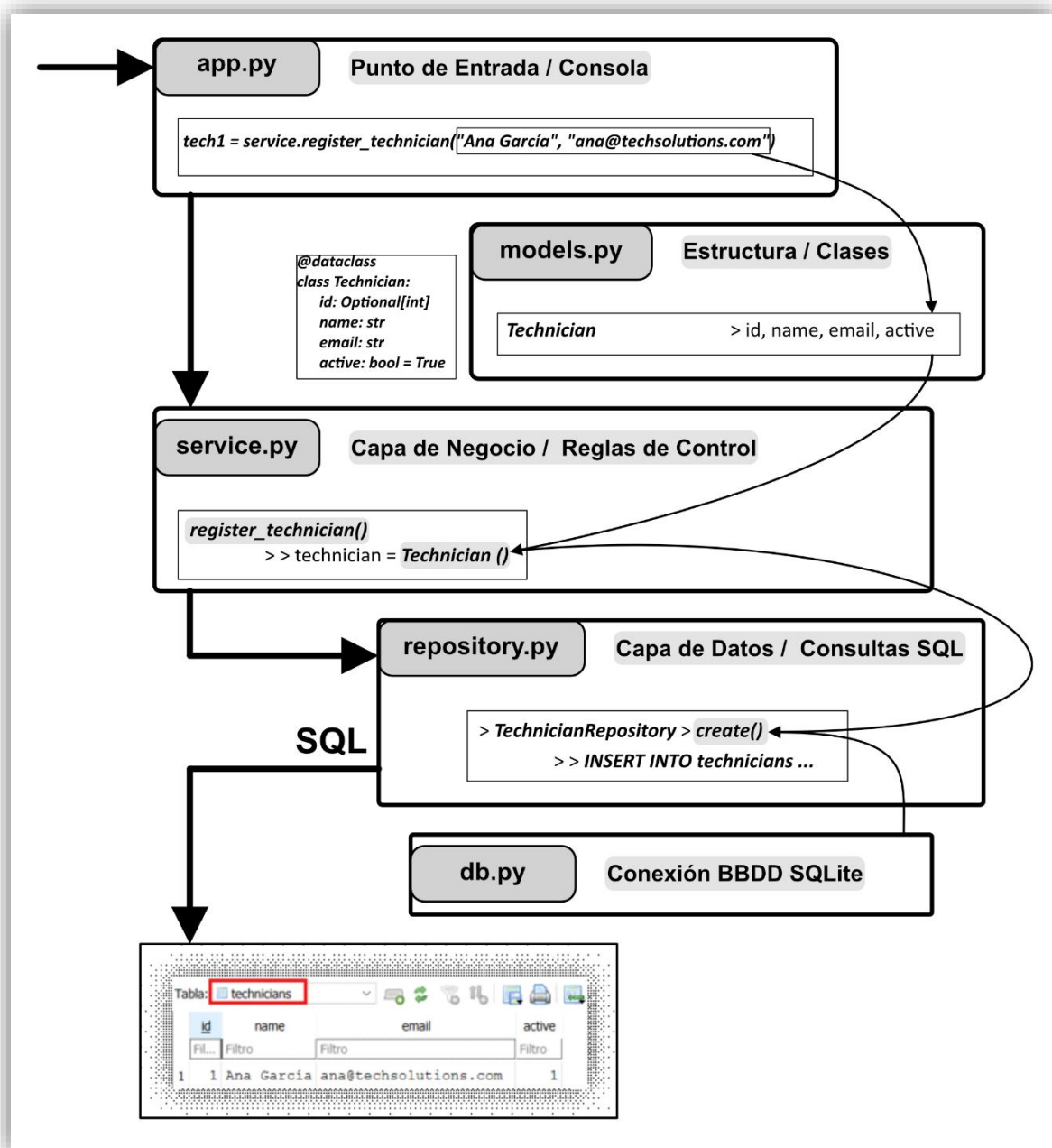
```
# Instanciación de la capa de Lógica de Negocio
service = Service()
```

> Crea un objeto **service** de la clase **Service**.

> Seguidamente ejecuta el **Paso 2**:

```
# -----
# PASO 2: Creación de Entidades (Técnicos)
# -----
print_step_header(2, "Registro de técnicos operativos en el sistema")
tech1 = service.register_technician("Ana García", "ana@techsolutions.com")
tech2 = service.register_technician("Luis Pérez", "luis@techsolutions.com")
print(f"> Entidad persistida: {tech1}")
print(f"> Entidad persistida: {tech2}")

pause_and_prompt()
```



```
> tech1 = service.register_technician("Ana García", "ana@techsolutions.com")
```

> Llama al método **register_technician** de la clase **Service** pasando como parámetros nombre y email.

> El método **register_technician** con esos parámetros crea un objeto de la clase **Technician** importada de **models.py** con **id = None**.

```
technician = Technician(  
    id = None,  
    name="Ana García",  
    email="ana@techsolutions.com"  
)
```

> Ese objeto se pasa al método **create** de la clase **TechnicianRepository** importado desde **repository.py**; este método último es el que hace la consulta SQL sobre la tabla **technicians** insertando los datos correspondientes en la tabla mediante sentencias preparadas, recuperando el índice (clave primaria que SQLite asigna a esa fila insertada) y devuelve un objeto de tipo **Technician** con los atributos **id**, **name**, **email**, **active**. Además guarda los cambios en la tabla. La sentencia SQL ejecutada equivaldría, en este caso, a **"INSERT INTO technicians (name, email, active) VALUES ('Ana García', 'ana@techsolutions.com', 1 if technician.active else 0)**.

> Cabe recordar que, por defecto, como se define en **models.py**, los objetos de clase **Technician** se instancian con **active: bool = True**

```
> tech2 = service.register_technician("Luis Pérez", "luis@techsolutions.com")
```

> De modo similar con un segundo registro.

```
--- PASO 2: Registro de técnicos operativos en el sistema ---  
> Entidad persistida: Technician(id=1, name='Ana García', email='ana@techsolutions.com', active=True)  
> Entidad persistida: Technician(id=2, name='Luis Pérez', email='luis@techsolutions.com', active=True)  
  
[Pausa] Operación finalizada. Presione [ENTER] para continuar al siguiente paso...|
```

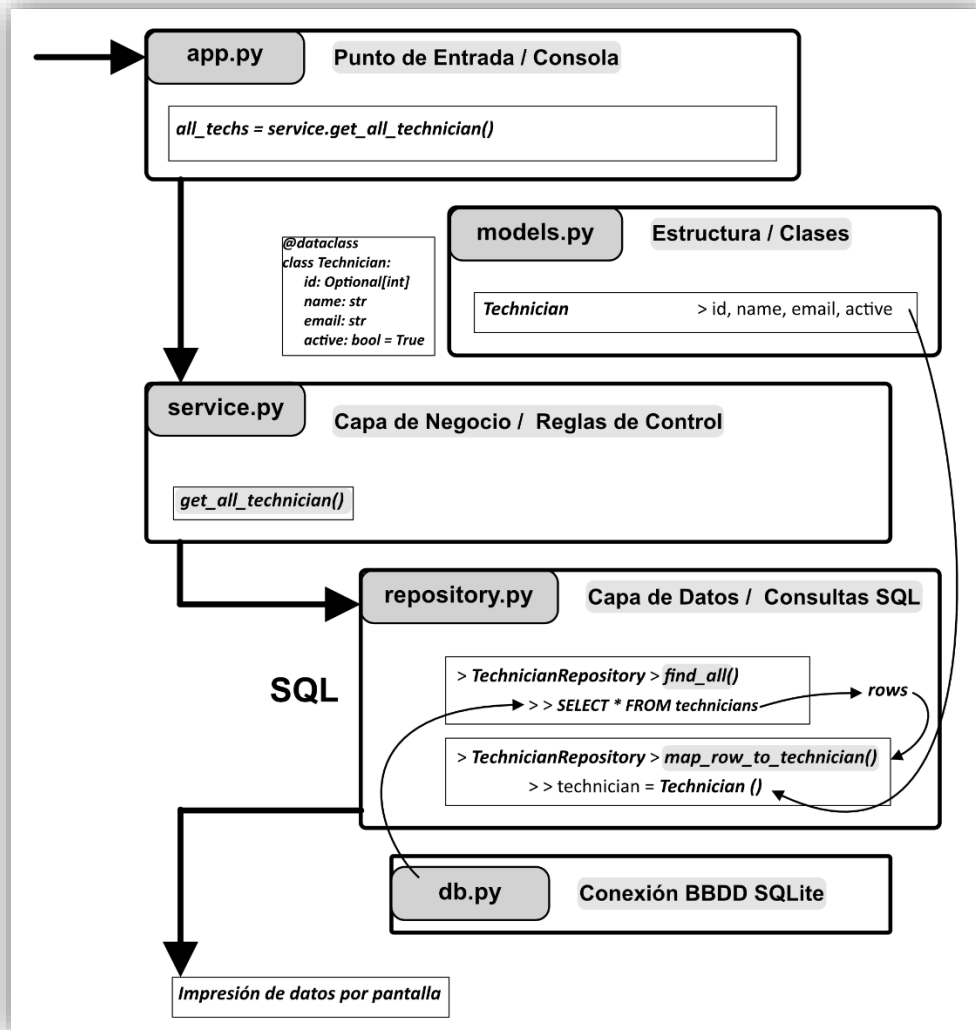
> Como resultado, la tabla **technicians** queda actualizada:

Tabla: technicians				
	id	name	email	active
	Fil...	Filtro	Filtro	Filtro
1	1	Ana García	ana@techsolutions.com	1
2	2	Luis Pérez	luis@techsolutions.com	1

> Seguidamente ejecuta el **Paso 3**:

```
# -----
# PASO 3: Lectura General (Técnicos)
# -----
print_step_header(3, "Consulta del listado completo de técnicos")
all_techs = service.get_all_technician()
for t in all_techs:
    print(f" - {t}")

pause_and_prompt()
```



> `all_techs = service.get_all_technician()`

> Llama al método `get_all_technician()` de la clase `Service`.

> El método `get_all_technician()` hace una llamada al método `find_all()` de la clase `TechnicianRepository` importado desde `repository.py`; este método último es el que hace la consulta SQL sobre la tabla `technicians`: `SELECT * FROM technicians`.

> El resultado de la consulta, que es un conjunto de filas, se pasa al método `map_row_to_technician` de la clase `TechnicianRepository` de forma iterativa fila a fila, para convertir los datos al modelo tipo definido para `Technician` en `models.py` (`id`, `name`, `email`, `active`) y poder mostrarlos por pantalla.

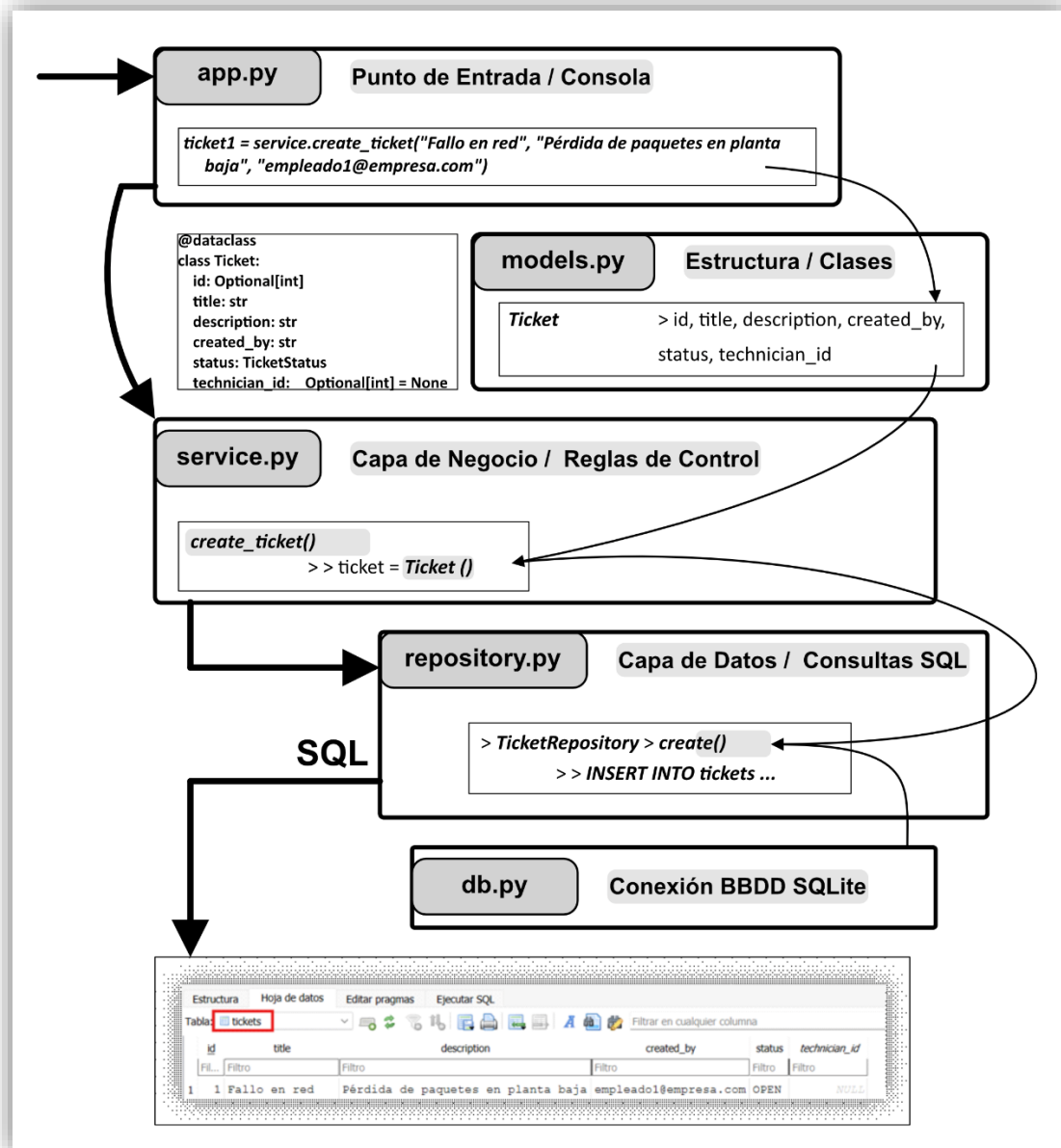
```
--- PASO 3: Consulta del listado completo de técnicos ---
- Technician(id=1, name='Ana García', email='ana@techsolutions.com', active=True)
- Technician(id=2, name='Luis Pérez', email='luis@techsolutions.com', active=True)

[Pausa] Operación finalizada. Presione [ENTER] para continuar al siguiente paso...|
```

> Seguidamente ejecuta el **Paso 4**:

```
# -----
# PASO 4: Creación de Entidades (Tickets)
# -----
print_step_header(4, "Registro de nuevas incidencias (Estado inicial: OPEN)")
ticket1 = service.create_ticket("Fallo en red", "Pérdida de paquetes en planta baja", "empleado1@empresa.com")
ticket2 = service.create_ticket("Hardware dañado", "Teclado sin respuesta", "empleado2@empresa.com")
print(f"> Entidad persistida: {ticket1}")
print(f"> Entidad persistida: {ticket2}")

pause_and_prompt()
```



En este caso se crean dos tickets.

> `ticket1 = service.create_ticket("Fallo en red", "Pérdida de paquetes en planta baja", "empleado1@empresa.com")`

> Llama al método `create_ticket` de la clase `Service` pasando como parámetros `title`, `description` y `created_by`.

> El método `create_ticket` con esos parámetros crea un objeto de la clase `Ticket` importada de `models.py` asignando un `id = None` y un `technician_id=None` (los tickets se crean sin id puesto que los asigna automáticamente SQLite y sin técnico asociado). Al mismo tiempo se crean los tickets con esta de incidencia "abierta" (`status=TicketStatus.OPEN`).

```
ticket = Ticket(
    id = None,
    title="Fallo en red",
    description="Pérdida de paquetes en planta baja",
    created_by="empleado1@empresa.com",
    status=TicketStatus.OPEN,
    technician_id=None
)
```

> Ese objeto se pasa al método `create` de la clase `TicketRepository` importado desde `repository.py`; este método último es el que hace la consulta SQL sobre la tabla `tickets` insertando los datos correspondientes en la tabla mediante sentencias preparadas, recuperando el índice (clave primaria que SQLite asigna a esa fila insertada) y devuelve un objeto de tipo `Ticket` con los atributos `id`, `title`, `email`, `description`, `created_by`, `status`, `technician_id`. Además guarda los cambios en la tabla.

La sentencia SQL ejecutada equivaldría, en este caso, a `"INSERT INTO tickets (title, description, created_by,status,technician_id) VALUES ('Fallo en red', 'Pérdida de paquetes en planta baja', 'empleado1@empresa.com', 'OPEN', None)"`.

> `ticket2 = service.create_ticket("Hardware dañado", "Teclado sin respuesta", "empleado2@empresa.com")`

> De modo similar con un segundo registro.

```
--- PASO 4: Registro de nuevas incidencias (Estado inicial: OPEN) ---
> Entidad persistida: Ticket(id=1, title='Fallo en red', description='Pérdida de paquetes en planta baja', created_by='empleado1@empresa.com', status=<TicketStatus.OPEN: 'OPEN'
> Entidad persistida: Ticket(id=2, title='Hardware dañado', description='Teclado sin respuesta', created_by='empleado2@empresa.com', status=<TicketStatus.OPEN: 'OPEN'>, techni

[Pausa] Operación finalizada. Presione [ENTER] para continuar al siguiente paso...
```

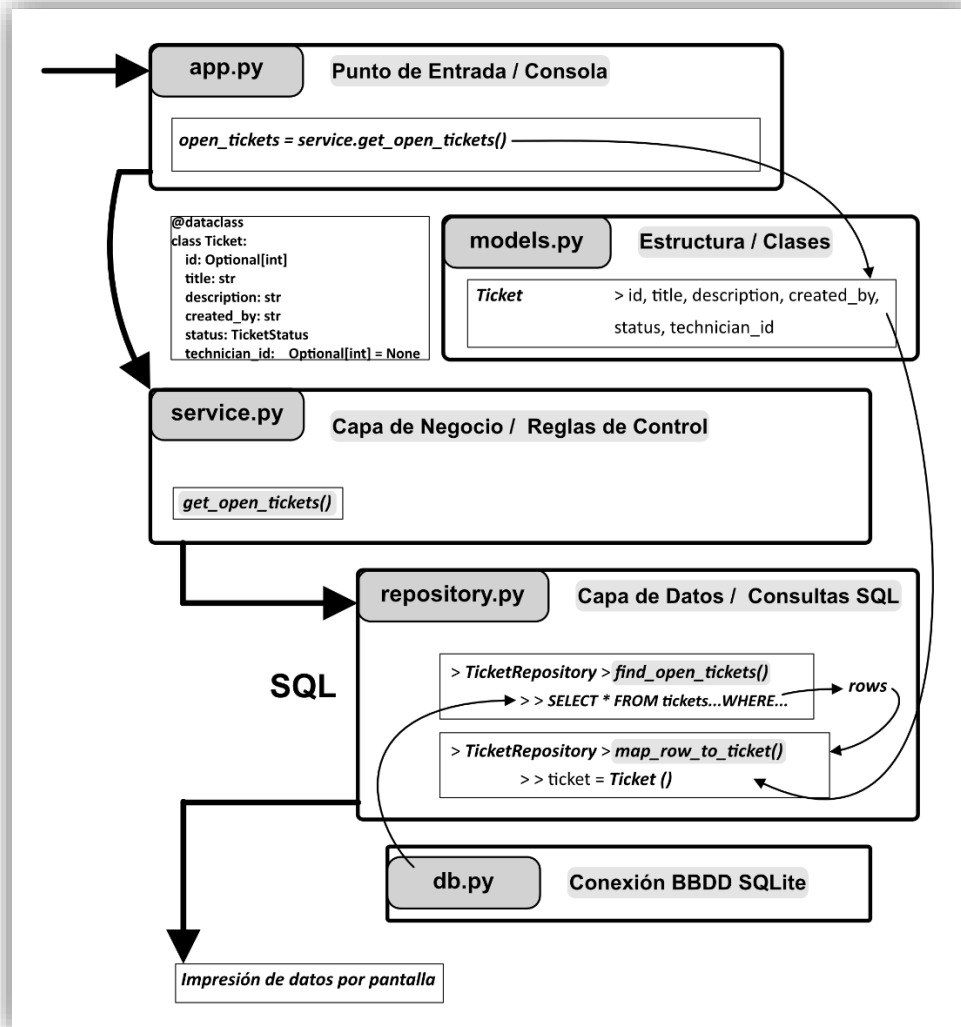
> Como resultado, la tabla `tickets` queda actualizada:

Estructura Hoja de datos Editar pragmas Ejecutar SQL						
Tabla: tickets		Filtrar en cualquier columna				
	id	title	description	created_by	status	technician_id
	Fil...	Filtro	Filtro	Filtro	Filtro	Filtro
1	1	Fallo en red	Pérdida de paquetes en planta baja	empleado1@empresa.com	OPEN	NULL
2	2	Hardware dañado	Teclado sin respuesta	empleado2@empresa.com	OPEN	NULL

> Seguidamente ejecuta el **Paso 5**:

```
# -----
# PASO 5: Lectura Filtrada (Tickets Abiertos)
# -----
print_step_header(5, "Consulta de incidencias pendientes (Tickets ABIERTOS)")
open_tickets = service.get_open_tickets()
for t in open_tickets:
    print(f" - {t}")

pause_and_prompt()
```



En este caso se consultan los tickets “abiertos”.

> **open_tickets = service.get_open_tickets()**

> Llama al método **get_open_tickets** de la clase **Service**.

> El método **get_open_tickets** hace una llamada al método **find_open_tickets** de la clase **TicketRepository** importado desde **repository.py**; este método último es el que hace la consulta SQL sobre la tabla **tickets**: **SELECT * FROM tickets WHERE status = "OPEN"**.

> El resultado de la consulta, que es un conjunto de filas, se pasa al método **map_row_to_ticket** de la clase **TicketRepository** de forma iterativa fila a fila, para convertir los datos al modelo tipo definido para **Ticket** en **models.py** (id, title, description, created_by, status, technician_id) y poder mostrarlos por pantalla.

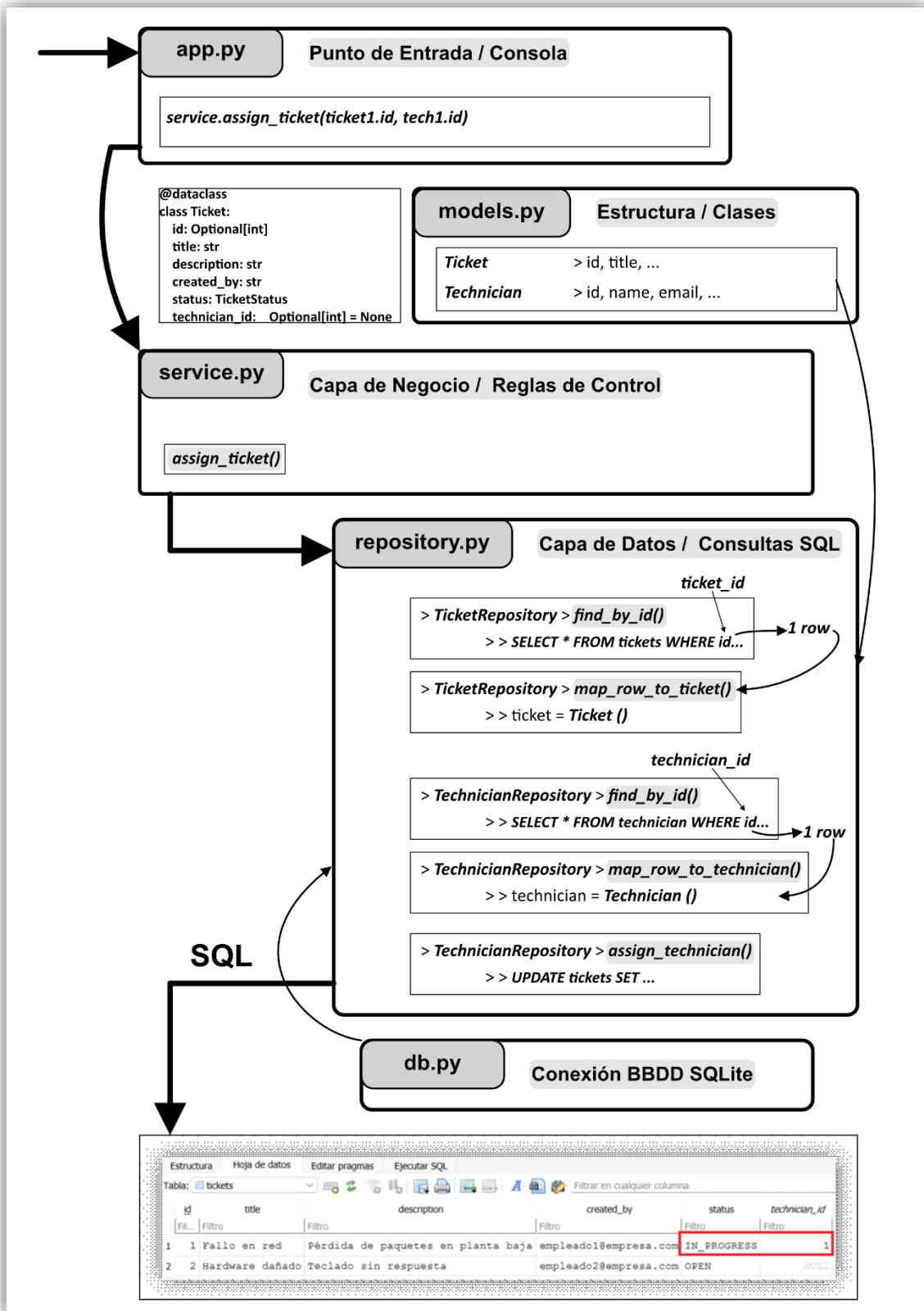
```
--- PASO 5: Consulta de incidencias pendientes (Tickets ABIERTOS) ---
1
2
- Ticket(id=1, title='Fallo en red', description='Pérdida de paquetes en planta baja', created_by='empleado1@empresa.com', status=<TicketStatus.OPEN: 'OPEN'>, technician_id=1)
- Ticket(id=2, title='Hardware dañado', description='Teclado sin respuesta', created_by='empleado2@empresa.com', status=<TicketStatus.OPEN: 'OPEN'>, technician_id=None)

[Pausa] Operación finalizada. Presione [ENTER] para continuar al siguiente paso...
```

> Seguidamente ejecuta el **Paso 6**:

```
# -----
# PASO 6: Modificación / Regla de Negocio (Asignación)
# -----
print_step_header(6, "Asignación de un técnico a una incidencia")
print(f"> Asignando el Ticket ID {ticket1.id} al Técnico ID {tech1.id}...")
service.assign_ticket(ticket1.id, tech1.id)
print("> Operación exitosa. El estado en base de datos debe ser ahora IN_PROGRESS y contener la clave foránea.")

pause_and_prompt()
```



En este caso se asigna un técnico a una incidencia.

> **`service.assign_ticket(ticket1.id, tech1.id)`**

> Llama al método **`assign_ticket`** de la clase **`Service`** pasando como parámetros **`ticket_id`** y **`technician_id`**.

> El método **`assign_ticket`** pasa el id del ticket buscado al método **`find_by_id`** de la clase **`TicketRepository`** importado desde **`repository.py`**; este método último es el que hace la consulta SQL sobre la tabla **`tickets`**, consulta que para los parámetros pasados se concreta en: **`SELECT * FROM tickets WHERE id = 1.`**

> El resultado de la consulta, que es una fila, se pasa al método **`map_row_to_ticket`** de la clase **`TicketRepository`** para convertir los datos al modelo tipo definido para **`Ticket`** en **`models.py`** (id, title, description, created_by, status, technician_id). El resultado se almacena en una variable **`ticket`**.

> Seguidamente el método **`assign_ticket`** llama al método **`assign_technician`** de la clase **`TechnicianRepository`** importado desde **`repository.py`**; este método último actualiza la tabla **`tickets`** realizando un UPDATE con los índices del ticket y técnico y es status. La consulta SQL en particular, para el ejemplo, se concreta en: **`UPDATE tickets SET technician_id = 1, status = "IN_PROGRESS" WHERE id = 1.`**

> El resultado es la actualización de la tabla **`tickets`** en la siguiente forma:

```
--- PASO 6: Asignación de un técnico a una incidencia ---
> Asignando el Ticket ID 1 al Técnico ID 1...
1
> Operación exitosa. El estado en base de datos debe ser ahora IN_PROGRESS y contener la clave foránea.

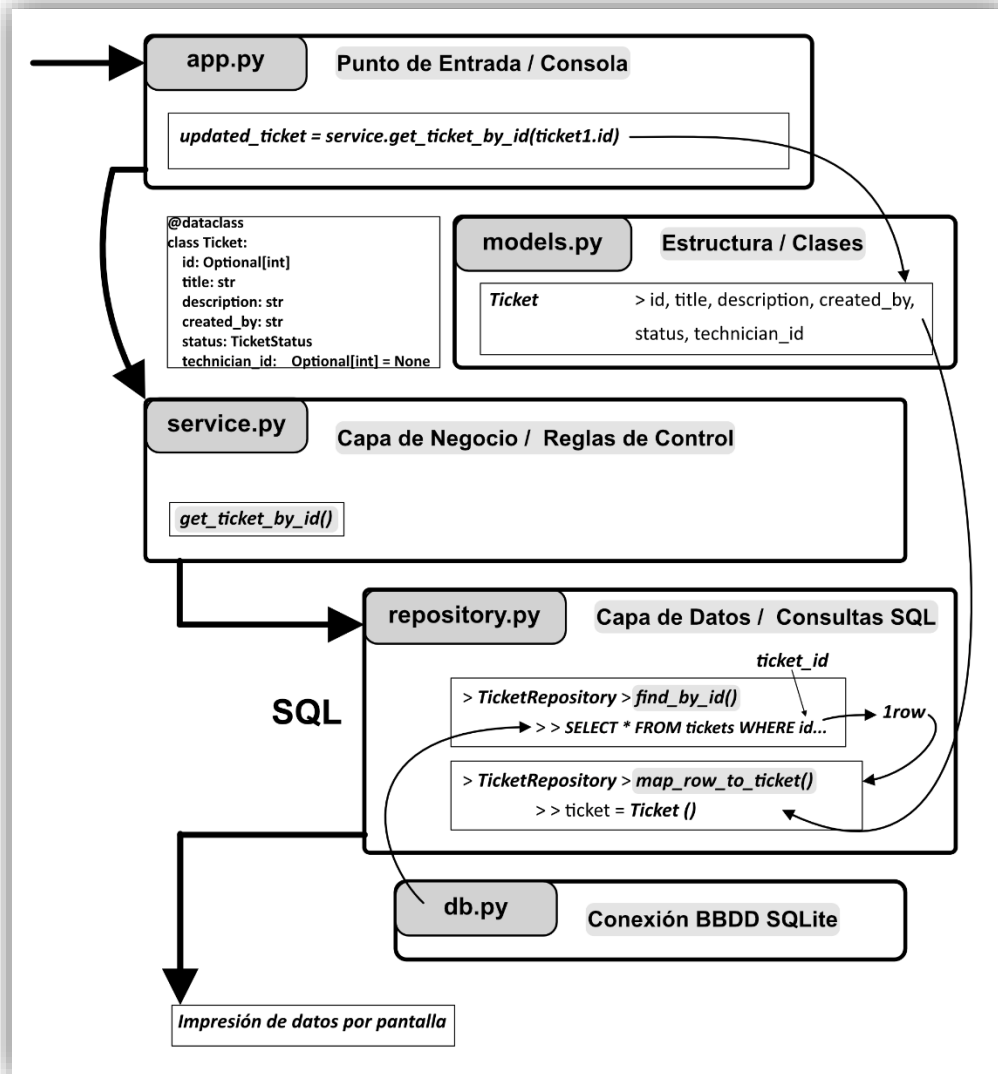
[Pausa] Operación finalizada. Presione [ENTER] para continuar al siguiente paso...
```

Estructura Hoja de datos Editar pragmas Ejecutar SQL						
Tabla: tickets						
id	title	description	created_by	status	technician_id	
Fil...	Filtro	Filtro	Filtro	Filtro	Filtro	
1	1 Fallo en red	Pérdida de paquetes en planta baja	empleado1@empresa.com	IN_PROGRESS	1	
2	2 Hardware dañado	Teclado sin respuesta	empleado2@empresa.com	OPEN		

> Seguidamente ejecuta el **Paso 7**:

```
# -----
# PASO 7: Búsqueda por Identificador
# -----
print_step_header(7, "Recuperación de entidad específica por ID")
updated_ticket = service.get_ticket_by_id(ticket1.id)
print(f"> Estado actual de la entidad modificada: {updated_ticket}")

pause_and_prompt()
```



En este caso se consulta el ticket modificado en el paso anterior.

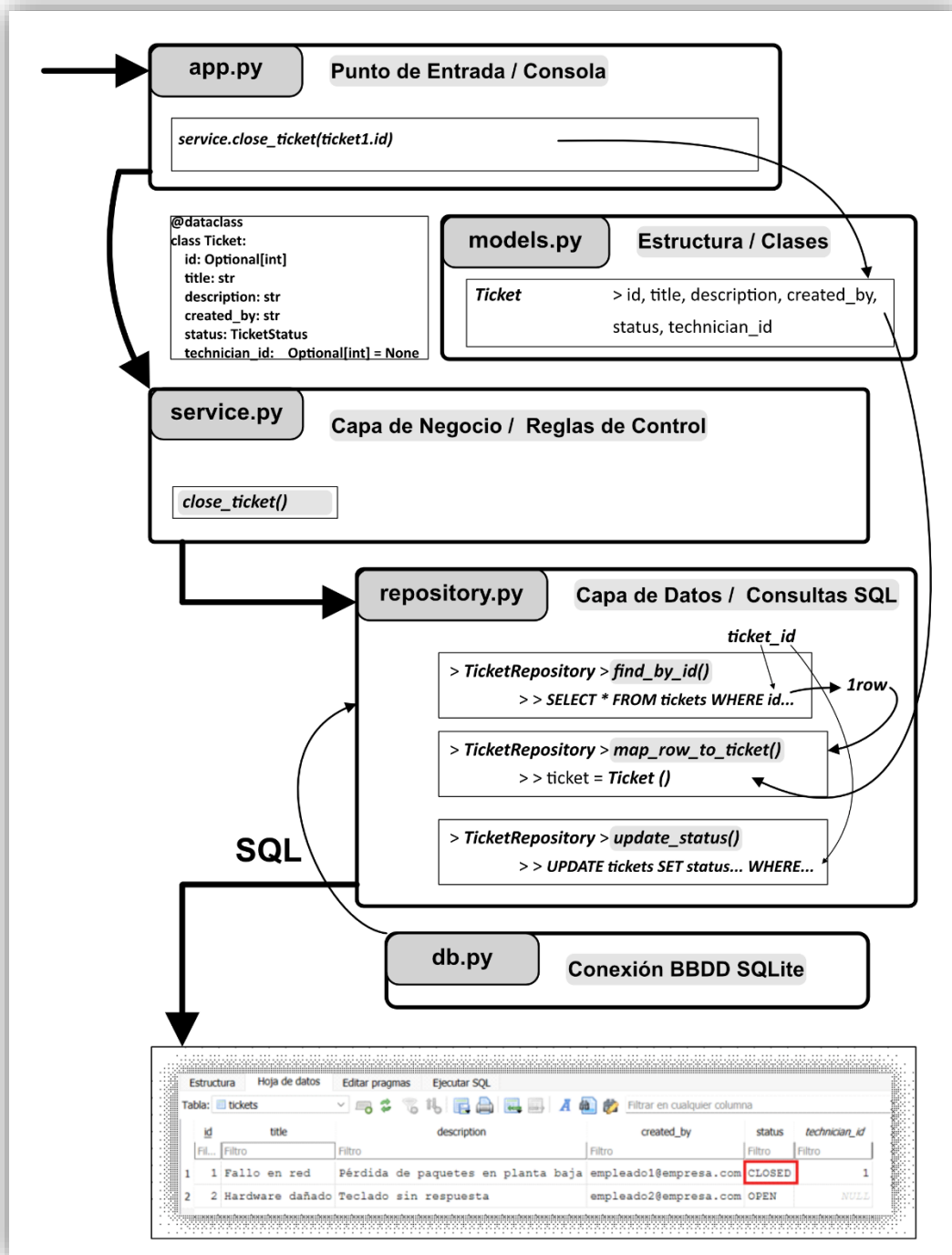
> **updated_ticket = service.get_ticket_by_id(ticket1.id)**

- > Llama al método **get_ticket_by_id** de la clase **Service**.
- > El método **get_ticket_by_id** hace una llamada al método **find_by_id** de la clase **TicketRepository** importado desde **repository.py**; este método último hace la consulta SQL contra la tabla **tickets**, tomando como parámetro el id del ticket buscado: **SELECT * FROM tickets WHERE id = 1**.
- > El resultado de la consulta, que es una fila, se pasa al método **map_row_to_ticket** de la clase **TicketRepository** de **models.py** para convertir los datos al modelo tipo definido para **Ticket** en **models.py** (id, title, description, created_by, status, technician_id) y poder mostrarlos por pantalla. El objeto es devuelto y se imprime por pantalla:

```
--- PASO 7: Recuperación de entidad específica por ID ---
1
> Estado actual de la entidad modificada: Ticket(id=1, title='Fallo en red', description='Pérdida de paquetes en planta baja', created_by='empleado1@empresa.com', status=<Tick
[Pausa] Operación finalizada. Presione [ENTER] para continuar al siguiente paso...
```

> Seguidamente ejecuta el **Paso 8**:

```
# -----  
# PASO 8: Modificación / Regla de Negocio (Cierre)  
# -----  
print_step_header(8, "Cierre de incidencia completada")  
print(f"> Procesando cierre para el Ticket ID {ticket1.id}...")  
service.close_ticket(ticket1.id)  
print("> Operación exitosa. El estado en base de datos debe ser ahora CLOSED.")  
  
pause_and_prompt()
```



En este caso se cierra la incidencia correspondiente al ticket 1.

> ***service.close_ticket(ticket1.id)***

> Llama al método ***close_ticket*** de la clase ***Service***, pasando como parámetro el id del ticket a encontrar (***ticket1.id***, que es ***1*** en este caso).

> El método ***close_ticket*** hace:

> Primero, una llamada al método ***find_by_id()*** de la clase ***TicketRepository***. Este ejecuta la búsqueda en SQL: ***SELECT * FROM tickets WHERE id = 1***. La búsqueda se guarda en una variable ticket para controlar si existe o no o si ya está cerrado.

> Seguidamente, hace una llamada al método ***update_status*** de la clase ***TicketRepository***. Al método se le pasan como parámetros el id del ticket y el nuevo status (1 y CLOSED) y ejecuta la sentencia SQL: ***UPDATE tickets SET status = "CLOSED" WHERE id = 1***.

```
--- PASO 8: Cierre de incidencia completada ---
> Procesando cierre para el Ticket ID 1...
1
> Operación exitosa. El estado en base de datos debe ser ahora CLOSED.

[Pausa] Operación finalizada. Presione [ENTER] para continuar al siguiente paso...
```

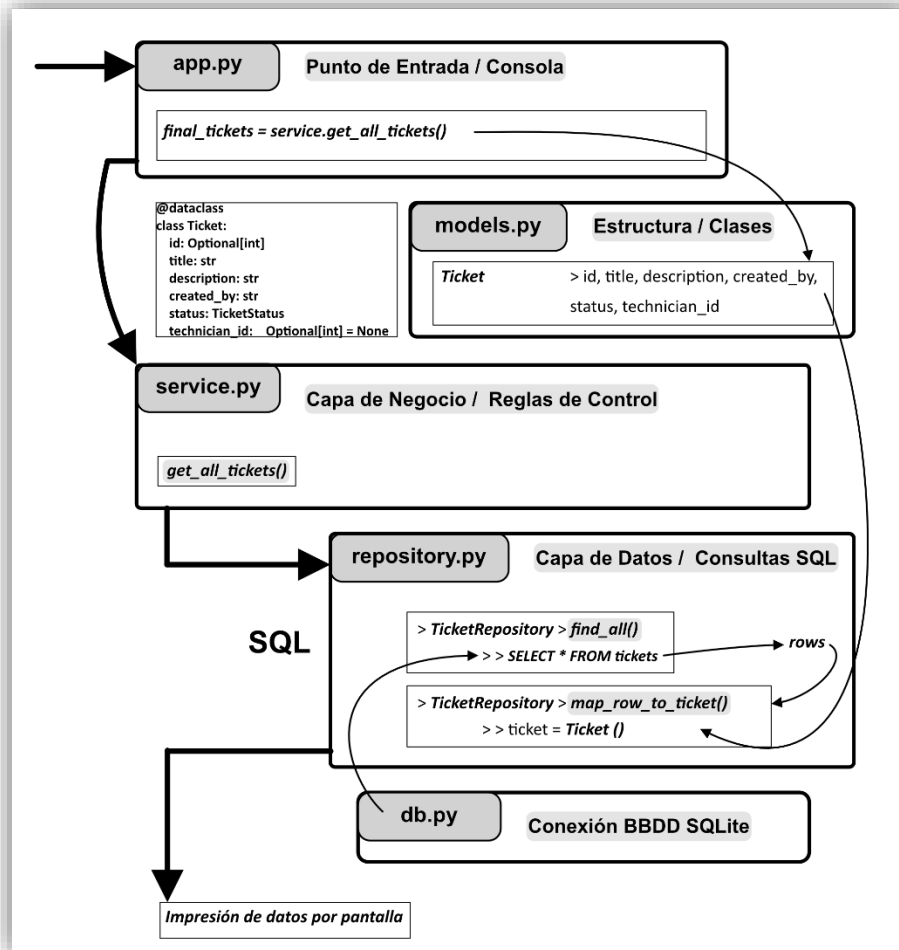
> La tabla ***tickets*** queda actualizada:

Estructura Hoja de datos Editar pragmas Ejecutar SQL						
Tabla: tickets						
	id	title	description	created_by	status	technician_id
	Fil...	Filtro	Filtro	Filtro	Filtro	Filtro
1	1	Fallo en red	Pérdida de paquetes en planta baja	empleado1@empresa.com	CLOSED	1
2	2	Hardware dañado	Teclado sin respuesta	empleado2@empresa.com	OPEN	NULL

> Seguidamente ejecuta el **Paso 9**:

```
# -----
# PASO 9: Lectura General y Verificación Final
# -----
print_step_header(9, "Consulta del listado completo de incidencias (Auditoría final)")
final_tickets = service.get_all_tickets()
for t in final_tickets:
    print(f" - {t}")

print("\n>>> Secuencia de pruebas completada. <<<")
```



Se consultan todos los tickets y su estado.

> `final_tickets = service.get_all_tickets()`

> Llama al método `get_all_tickets()` de la clase **Service**.

> El método `get_all_tickets()` hace una llamada al método `find_all()` de la clase **TicketRepository**. Este método ejecuta la búsqueda en SQL: `SELECT * FROM tickets` y la guarda para posteriormente recorrer fila a fila pasando cada una al método `map_row_to_ticket` de **TicketRepository**, convirtiendo cada fila de datos a un objeto de modelo tipo definido para **Ticket** en `models.py` (`id`, `title`, `description`, `created_by`, `status`, `technician_id`). Finalmente devuelve todos estos objetos en forma de lista.

> La lista se recorre y se muestra cada elemento por pantalla.

```
--- PASO 9: Consulta del listado completo de incidencias (Auditoría final) ---
1
2
- Ticket(id=1, title='Fallo en red', description='Pérdida de paquetes en planta baja', created_by='empleado1@empresa.com', status=<TicketStatus.CLOSED: 'CLOSED'>, technician_id=1)
- Ticket(id=2, title='Hardware dañado', description='Teclado sin respuesta', created_by='empleado2@empresa.com', status=<TicketStatus.OPEN: 'OPEN'>, technician_id=None)

>>> Secuencia de pruebas completada. <<<

Process finished with exit code 0
```

4. COMPLETANDO EL CÓDIGO (Requerimientos del enunciado)

4.1 Implementando el código

En el código realizado en clase faltan por implementar algunas condiciones de las requeridas en el enunciado; se trata de **3 reglas de negocio**, todas pertenecientes a la **Capa de Lógica de Negocio** y deben insertarse en el archivo **service.py**:

- **Validación de longitud de título de un ticket:** un ticket no puede crearse si su título está **vacío** o compuesto solo por espacios en blanco. (Ubicación: **service.py** → método **create_ticket**).
- **Validación de formato de correo:** el campo **created_by** debe contener una **dirección de correo válida** (mínimo, debe incluir el carácter **@**). (Ubicación: **service.py** → método **create_ticket**).
- **Dependencia de asignación para el cierre:** un ticket no puede cambiar su estado a **CLOSED** si no tiene un **técnico asignado previamente**. (Ubicación: **service.py** → método **close_ticket**).

> Como todas han de implementarse en el archivo "**service.py**" se realiza una copia del original que nombramos "**service2.py**" para incluir este código como se detalla a continuación:

> Modificación del método **create_ticket**, que queda como sigue:

```
def create_ticket(self, title: str, description: str, created_by: str) -> Ticket:
    # 1. Validar que el título no esté vacío ni sea solo espacios
    if not title or len(title.strip()) == 0:
        raise ValueError("Regla de negocio fallida: El título del ticket no puede estar vacío.")

    # 2. Validar que el correo del creador contenga '@'
    if "@" not in created_by:
        raise ValueError("Regla de negocio fallida: El email del creador debe contener el carácter '@'")

    # 3. Creación de la entidad si pasa las validaciones
    ticket = Ticket(
        id=None,
        title=title,
        description=description,
        created_by=created_by,
        status=TicketStatus.OPEN,
        technician_id=None
    )

    return self.ticket_repository.create(ticket)
```

> **Análisis** de la implementación:

- **not title:** atrapa el error si por algún motivo la capa superior envía un valor nulo (None).
- **len(title.strip()) == 0:** > la función **.strip()** elimina los espacios en blanco al principio y al final del texto. Si después de quitar los espacios la longitud es 0 (es decir, el usuario envió algo como " "), el sistema lanza un **ValueError** y bloquea la creación, protegiendo la base de datos de entradas inútiles.
- **"@" not in created_by:** es una validación estructural básica y muy eficiente. Si la cadena de texto no posee una **@**, interrumpe el flujo y exige un formato de email correcto.

> Modificación del método `close_ticket`, que queda como sigue:

```
def close_ticket(self, ticket_id: int) -> None:
    ticket = self.ticket_repository.find_by_id(ticket_id)

    # Validaciones de existencia y estado (Ya estaban correctas)
    if ticket is None:
        raise ValueError(f"El ticket {ticket_id} no existe")
    if ticket.status == TicketStatus.CLOSED:
        raise ValueError("El ticket ya se encuentra cerrado")

    # NUEVA REGLA: Verificar que haya un técnico asignado
    if ticket.technician_id is None:
        raise ValueError("Regla de negocio fallida: No se puede cerrar un ticket sin un técnico asignado.")

    # Persistencia en base de datos
    update = self.ticket_repository.update_status(ticket_id, TicketStatus.CLOSED)
    if not update:
        raise ValueError("No se pudo cerrar el ticket debido a un error en la base de datos")
```

> **Análisis** de la implementación:

- **if ticket.technician_id is None:** > una vez que el sistema ha recuperado la entidad de la base de datos, se inspecciona su atributo `technician_id`. Conforme al modelo (**models.py**) que si un ticket está recién creado y no ha pasado por el método `assign_ticket`, este campo es `None`.
- **Cláusula de guarda:** al colocar este `if` justo antes de ejecutar `self.ticket_repository.update_status(...)`, se garantiza que la orden de cierre nunca llegue al motor SQL si la incidencia no ha sido atendida por un profesional, cumpliendo estrictamente con el diagrama de flujo operativo de la empresa.

4.2 Comprobaciones adicionales

Para comprobar que las nuevas reglas de negocio funcionan, no se pueden simplemente ejecutar los comandos sueltos. Como se ha configurado la capa de servicio para que lance una excepción (**raise ValueError**) cuando se incumple una regla, si no se prepara el archivo de pruebas para "atrapar" ese error, el programa colapsará y se cerrará abruptamente en el primer fallo.

Para solucionar esto, se usa la estructura de control **try...except**. Esta estructura intenta (**try**) ejecutar una acción; si la acción falla y lanza un error, en lugar de colapsar la aplicación, captura la excepción (**except**) y nos permite decidir qué hacer (por ejemplo, imprimir un mensaje de éxito indicando que el bloqueo funcionó).

Por tanto, para la prueba se genera un nuevo archivo llamado **"test_reglas.py"** diseñado para forzar los errores y confirmar que el sistema se defiende correctamente. El código es el siguiente:

```
import os
from service2 import Service
from db import init_db

def test_reglas_negocio() -> None:
    print("=" * 60)
    print("INICIANDO AUDITORÍA DE REGLAS DE NEGOCIO (RUTAS NEGATIVAS)")
    print("=" * 60 + "\n")

    # 0. Preparación del entorno
    if not os.path.exists("claseExperiencial.db"):
        init_db()
    service = Service()

    # -----
    # PRUEBA 1: Título vacío o con espacios
    # -----
    print("--- PRUEBA 1: Título de incidencia vacío ---")
    try:
        # Intentamos enviar un título compuesto solo por espacios
        service.create_ticket(" ", "El teclado no funciona", "usuario@empresa.com")
        print("❌ FALLO DE SEGURIDAD: El ticket se creó ignorando la regla.")
    except ValueError as error:
        # Si entra aquí, significa que nuestro 'raise ValueError' funcionó
        print(f"✅ BLOQUEO EXITOSO: {error}\n")

    # -----
    # PRUEBA 2: Formato de email inválido
    # -----
    print("--- PRUEBA 2: Email sin carácter especial (@) ---")
    try:
        # Intentamos enviar un email sin el arroba
        service.create_ticket("Fallo de red", "No hay internet", "usuario.empresa.com")
        print("❌ FALLO DE SEGURIDAD: El ticket se creó ignorando la regla.")
    except ValueError as error:
        print(f"✅ BLOQUEO EXITOSO: {error}\n")

    # -----
    # PRUEBA 3: Cierre de ticket sin técnico asignado
    # -----
    print("--- PRUEBA 3: Cerrar incidencia no asignada ---")
    try:
        # Primero, creamos un ticket válido para tener un registro real sobre el que operar
        ticket_valido = service.create_ticket("Pantalla azul", "El PC no arranca", "admin@empresa.com")
        print(f"> (Preparación) Ticket ID {ticket_valido.id} creado con éxito.")
        print(f"> Intentando forzar el cierre del Ticket ID {ticket_valido.id}...")

        # Ahora, intentamos cerrarlo inmediatamente, SIN invocar a 'assign_ticket' previamente
        service.close_ticket(ticket_valido.id)

        print("❌ FALLO DE SEGURIDAD: El ticket se cerró saltándose el flujo operativo.")
    except ValueError as error:
        print(f"✅ BLOQUEO EXITOSO: {error}\n")

if __name__ == "__main__":
    test_reglas_negocio()
```

> Al *ejecutar el código* observamos:

```
C:\Users\fcojg\PycharmProjects\PythonProject4\.venv\Scripts\python.exe C:\Users\fcojg\PycharmProjects\PythonProject4\IncidenciaTickets\test_reglas.py
=====
INICIANDO AUDITORÍA DE REGLAS DE NEGOCIO (RUTAS NEGATIVAS)
=====

--- PRUEBA 1: Título de incidencia vacío ---
✅ BLOQUEO EXITOSO: Regla de negocio fallida: El título del ticket no puede estar vacío.

--- PRUEBA 2: Email sin carácter especial (@) ---
✅ BLOQUEO EXITOSO: Regla de negocio fallida: El email del creador debe contener el carácter '@'.

--- PRUEBA 3: Cerrar incidencia no asignada ---
> (Preparación) Ticket ID 4 creado con éxito.
> Intentando forzar el cierre del Ticket ID 4...
4
✅ BLOQUEO EXITOSO: Regla de negocio fallida: No se puede cerrar un ticket sin un técnico asignado.

Process finished with exit code 0
```

> El programa **no se detiene** a pesar de los errores.

> La **base de datos permanece intacta** para las pruebas 1 y 2 (no se registrará ninguna basura en las tablas).

> La **prueba 3** muestra cómo la capa de servicio **rechaza un comando de modificación** de estado protegiendo la coherencia de los datos en SQLite.

> Notas

> **raise ValueError**

La instrucción **raise ValueError("Mensaje")** constituye el mecanismo estándar de Python para la **generación y lanzamiento explícito de excepciones** (errores controlados) durante la ejecución de un programa. Su función principal es alertar de que se ha producido una condición anómala que impide continuar con el flujo normal de las operaciones.

Componentes:

- **raise**: palabra reservada (*keyword*) del lenguaje Python.
Su ejecución **interrumpe inmediatamente el flujo** secuencial del código en esa línea. Actúa como un disparador que transfiere el control de la aplicación al sistema de gestión de errores.
- **ValueError**: clase de excepción **nativa** (incorporada por defecto en Python).
Semánticamente, se utiliza para indicar que una función o método ha **recibido un argumento que tiene el tipo de dato correcto** (por ejemplo, un número entero), **pero cuyo valor es inapropiado** o lógicamente inaceptable para la operación (por ejemplo, solicitar el identificador de un técnico que no existe en la base de datos).
- **("Mensaje")**: **argumento** de instanciación de la clase ValueError.
Consiste en una cadena de texto descriptiva que **documenta el motivo exacto del fallo**. Esta información es crucial para la depuración (*debugging*) o para informar a la capa superior (interfaz de usuario) sobre la causa del rechazo.

Mecanismo de acción y flujo de ejecución:

1. **Interrupción**: en el instante en que el intérprete de Python procesa la instrucción **raise**, la **función actual se aborta por completo**. Ninguna línea de código situada por debajo del raise dentro de ese bloque será ejecutada.
2. **Propagación en la pila de llamadas**: el **error** generado inicia un **ascenso** automático a través de las capas de la aplicación (hacia la función que invocó a la actual).
3. **Captura o Colapso**: la excepción continuará **propagándose hacia arriba hasta** encontrar una estructura de control **try...except** diseñada para interceptarla y gestionarla de forma segura.
Si alcanza la capa superior sin ser capturada (como ocurre actualmente en el archivo app.py), el programa **colapsa**, detiene su ejecución y **muestra** en consola un volcado de pila (**Traceback**) con el mensaje original.

Propósito arquitectónico en la Capa de Servicio:

En el contexto del archivo **service.py**, la estructura **raise ValueError** se emplea como un mecanismo de validación defensiva, conocido como **cláusula de guarda (Guard Clause)**.

Permite **aislar y proteger la base de datos**.

Si una regla de negocio se incumple (por ejemplo, intentar cerrar un ticket ya cerrado), el **raise ValueError** bloquea la ejecución de forma prematura y categórica, garantizando que la instrucción de actualización (UPDATE) en el repositorio nunca llegue a ejecutarse, **preservando así la integridad relacional de los datos**.